



西安邮电大学

本科毕业设计（论文）英文翻译

学 生 姓 名 :	杨佳星
学 号 :	04222088
班 级 :	2203
专 业 :	网络工程
院 （系） :	计算机学院
指 导 教 师 :	王晓梅

二零二六年六月

一、英文原文

英文来源: Pahl C, Brogi A, Soldani J, Jamshidi P. Cloud Container Technologies: A State-of-the-Art Review[J]. IEEE Transactions on Cloud Computing, 2019, 7(3): 677-692.

起止页码: 677-692

出版日期(期刊号): 2019 年 7 月, Vol. 7, No. 3

刊物名称: IEEE Transactions on Cloud Computing (IEEE 云计算汇刊)

DOI: 10.1109/TCC.2017.2702586

Cloud Container Technologies: a State-of-the-Art Review

Claus Pahl* and Antonio Brogi ‡ and Jacopo Soldani ‡ and Pooyan Jamshidi †

* Faculty of Computer Science, Free University of Bozen-Bolzano, Bolzano, Italy ‡
Department of Computer Science, University of Pisa, Pisa, Italy † School of Computer Science,
Carnegie Mellon University, Pittsburgh, USA

Abstract-Containers as a lightweight technology to virtualise

applications have recently been successful, particularly to manage applications in the cloud. Often, the management of clusters of containers becomes essential and the orchestration of the construction and deployment becomes a central problem. This emerging topic has been taken up by researchers, but there is currently no secondary study to consolidate this research. We aim to identify, taxonomically classify and systematically compare the existing research body on containers and their orchestration and specifically the application of this technology in the cloud. We have conducted a systematic mapping study of 46 selected studies. We classified and compared the selected studies based on a characterisation framework. This results in a discussion of agreed and emerging concerns in the container orchestration space, positioning it within the cloud context, but also moving it closer to current concerns in cloud platforms, microservices and continuous development.

Keywords: Orchestration, Container, Cluster, Cloud, Systematic Literature Review, Systematic Mapping Study.

I. INTRODUCTION

Containerisation is a technology to virtualise applications in a lightweight way that has resulted in a significant uptake in cloud applications management. How to orchestrate the construction and deployment of containers individually and in clusters has become a central problem (Pahl, 2015). There has not been a secondary study of research on container technologies in the cloud that would allow to assess the maturity in general and identify trends, research gaps

and future directions. Given the growing interest in containers, their management and orchestration in cloud, there is a need to explore current research. Secondary studies identify, classify and synthesise a comparative overview of state-of-the-research and enable an assessment of ongoing work (Petersen et al., 2008; Kitchenham et al., 2009). We opt for a systematic mapping study (SMS) as it is more suitable in mapping out and structuring new areas of investigation. We identify, taxonomically classify and systematically compare the existing research body on container technologies and its application in the cloud, aiming to extract a better understanding of Platform-as-a-Service (PaaS) as middleware built on containers for application packaging and as a deployment infrastructure. We have conducted a systematic mapping study of 46 selected studies (Table II), spanning over a decade from 2007 onwards. We classified and compared the selected studies based on a characterisation framework. Our mapping study resulted in a knowledge base of current research approaches, methods, techniques, best practices and experiences used in cloud architecture, with a particular attention to cloud application development and management. Our study revealed that container technologies research is still in a formative stage. More experimental and empirical evaluation of benefits is needed. Our study also showed a lack of tool support to automate and facilitate container management and orchestration, specifically in clustered cloud architectures. The results of our mapping study show growing interests and usage of container-based technologies (such as LXC or Docker) as lightweight virtualisation solutions at Infrastructure-as-a-Service (IaaS) level, and as application management solutions at PaaS level. We can observe that containers positively impact on both development and deployment aspects. For instance, architecting in the cloud moves towards DevOps-based approaches, supporting a continuous development and deployment pipeline taking into account cloud-native architecture solutions based on containers and their orchestration (Brunnert et al., 2015). The results show that containers can support continuous development in the cloud based on cloud-native platform services for development and deployment, but do require advanced orchestration support. Container-based orchestration techniques hence emerge as a mechanism to orchestrate computation in cloud-based, clustered environments. The results of our study show that such techniques are seen to balance the need of technical quality management, e.g., optimised resource utilisation and performances, which is a cost factor in the cloud (due to its utility pricing principle). Our systematic mapping study aims to benefit, firstly, researchers in software engineering, distributed systems and cloud computing, who need an identification of relevant studies. A systematic presentation of research provides a body of knowledge to develop theory and solutions, analyse research implications and establish future dimensions. It also benefits practitioners interested in understanding the available methods, techniques and tools as well as their constraints and maturity level. This paper is structured as follows. Section II describes background and related research to position this work. Section III explains the research methodology, research questions

and scope. Section IV provides a characterisation framework for cloud container orchestration. Section V presents the results of the mapping study, followed by an analysis of its limitations.

Section VI discusses findings, implications and trends.

II. CONTAINER ARCHITECTURES AND THEIR

MANAGEMENT The cloud uses virtualisation techniques to achieve elasticity of large-scale shared resources (Mell and Grance, 2011). Virtual machines (VMs) are typically the backbone at the infrastructure layer. Containerisation in contrast allows a lightweight virtualisation through the bespoke construction of containers as application packages from individual images (generally retrieved from an image repository) that consume less resources and time. They also support a more interoperable application packaging needed for portable, interoperable software applications in the cloud (Pahl, 2015). Containerisation is based on the capability to develop, test and deploy applications to a large number of servers and also to interconnect these containers. Containers address consequently concerns at the cloud PaaS level. Given the overall importance of the cloud, a consolidating view on current activities is important.

A. Container Technology Principles

A container holds packaged self-contained, ready-to-deploy parts of applications and, if necessary, middleware and business logic (in binaries and libraries) to run the applications. Tools like Docker are built around container engines where containers act as portable means to package applications. This results in the need to manage dependencies between containers in multi-tier applications. An orchestration plan can describe components, their dependencies and their lifecycle in a layered plan. A PaaS cloud can then execute the workflows from the plan through agents (like a container engine). PaaS clouds can consequently support the deployment of applications from containers. Orchestration subsumes here their coordinated construction, deployment and ongoing management (Liu et al., 2011). Many container solutions are based on Linux LXC techniques. Recent Linux distributions - part of the Linux container project LXC - provide kernel mechanisms such as namespaces and cgroups to isolate processes on a shared operating system [S5]. Docker is the most popular container solution at the moment and shall be used to illustrate containerisation. A Docker image is made up of file systems layered over each other, similar to the Linux virtualisation stack, using the LXC mechanisms, see Fig. 1 (right). Docker uses a union mount to add a writable file system on top of the read-only file system. This allows multiple read-only file systems to be stacked on top of each other. This property can be used to create new images by building on top of base images. Only the top layer is writable, which is the container itself. Containerisation facilitates the step from single applications in containers to clusters of container hosts that can run containerised applications across cluster hosts [S9]. The latter benefits from the built-in interoperability of containers. Individual container hosts are

grouped into interconnected clusters, illustrated in Fig. 1 (left). Each cluster consists of several (host) nodes. Application services are logical groups of containers from the same image. Application services allow scaling an application across different host nodes. Volumes are mechanisms used for applications that require data persistence. Containers can mount these volumes for storage. Links allow two or more containers to connect and communicate. The set-up and management of these container clusters requires orchestration support for inter-container communication, links and service assemblies (Pahl, 2015).

Fig. 1: Container Cluster Architectures.

B. Cloud-based Container Architectures

Container orchestration deals not only with turning applications on or off (i.e., start or stop containers) and moving them among servers. We define orchestration as constructing and continuously managing possibly distributed clusters of container-based software applications. Container orchestration allows users to define how to coordinate the containers in the cloud when a multi-container application is deployed. Container orchestration defines not only the initial deployment of containers, but also the management of the multi-containers as a single entity. It takes care of availability, scaling and networking of containers. Essentially cloud-based container construction is a form of orchestration within the distributed cloud environment. The cloud can be seen as a distributed and tiered architecture, see Fig. 2, with core infrastructure, platform and software application tiers distributed across multi-cloud environments (Brogi et al., 2016). Container technologies can help. As such, container technologies will play a central role in the future of application management, in particular in the cloud PaaS context.

Fig. 2: Cloud Reference Architecture Model.

Recently popular microservice-based architectures can be realised in this cloud framework through containers (Lewis and Fowler, 2014; Kratzke, 2015). Given this change in

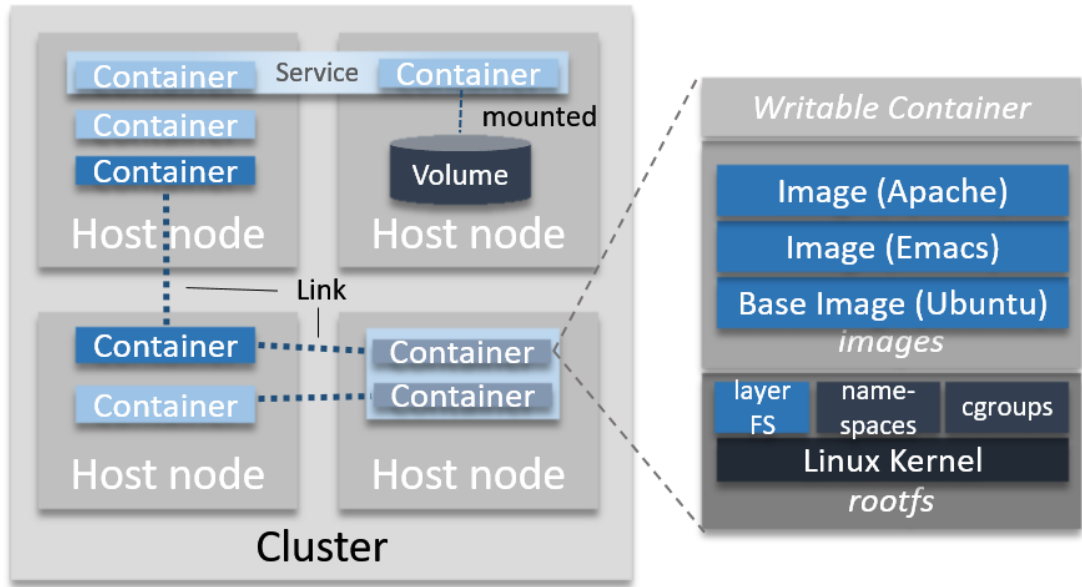


图 1: 容器集群架构。

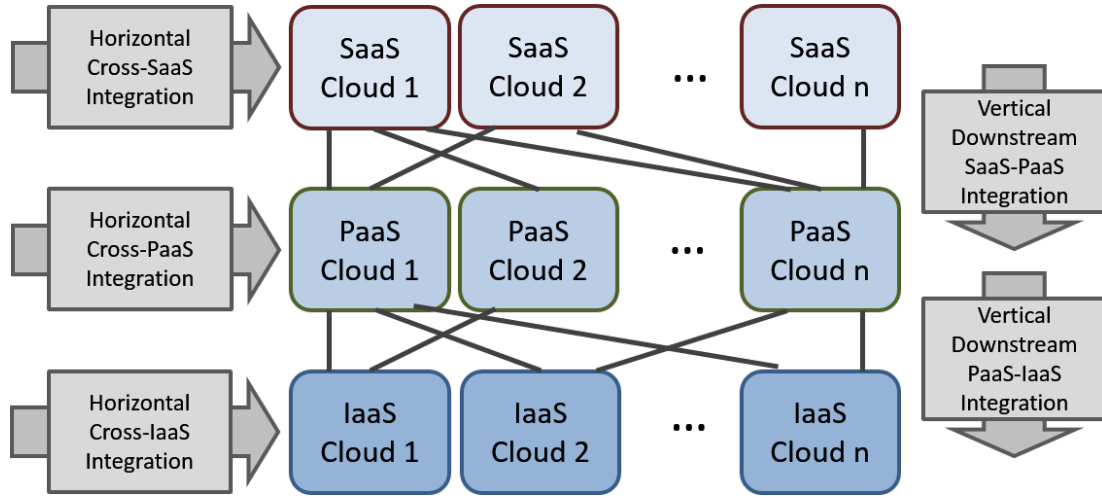


图 2: 云参考架构模型。

architecting, a secondary study can help practitioners for their decision making in terms of the correct technology choice.

C. State-of-the-Art

The mechanism we use to review cloud container techniques is that of a systematic mapping study. Reviews can be distinguished into two forms. Systematic Literature Reviews (SLR) suit summative analyses of mature fields, based on possibly larger bodies of literature. Systematic Mapping Studies (SMS) are suitable to determine the structure of the type of research reports, visual categorisation, useful if there is a lack of highquality primary studies. SMSs are typically less detailed, but they more appropriate for our purposes. As part of our paper selection process, we extracted six review papers that are related to our aim (see Table II). Five out of these six qualify as technology reviews, i.e., they overview and assess container technologies. Study [S3] covers virtualisation basics and container construction/management. The focus is more on

deployment than development. Study [S4] clearly addresses virtualisation basics only, but from deployment and development perspectives. Study [S6] is then more comprehensive, including clusters and both deployment and development perspectives. Study [S9] is similar to [S6], but has less quality management concerns covered. Study [S19] focusses like [S4] on virtualisation basics, but specifically on performance in HPC and computation/storage intensive applications. Slightly different in the approach is study [S17], which is more organised around research coverage rather than only technology. However, a systematic coverage of literature (like the one we propose in this paper) is missing. SLRs and SMSs entail to identify, classify and compare existing evidence on the use of container technologies specifically in cloud environments through a characterisation framework. As highlighted above, some technology reviews exist, but these concentrate on technology and do not capture research efforts and directions systematically.

III. RESEARCH METHODOLOGY

A. A Systematic Mapping Process

SMSs reduce bias through a rigorous sequence of methodological steps to search and classify literature. They rely on well-defined and evaluated review protocols to extract, analyse and document results. We follow the process presented in (Petersen et al., 2008) with a three-step review that includes planning, conducting and documenting. Review Step Activity plan identify need, specify research questions, define protocol conduct select primary studies, extract/synthesise data document document observations, analyse threats, report The review is complemented by an evaluation of each step's outcome. Furthermore, we provide an additional characterisation framework for the study context. We have adapted and applied a systematic mapping to cloud technology in a study focusing on container orchestration. The essential process steps of our systematic mapping study are definition of research questions, conducting the search for relevant papers, screening of papers, keywording of abstracts and data extraction and mapping. Each process step has an outcome, the final outcome of the process being the systematic map: Process Steps Outcomes Section Definition of Research Question Review Scope III-B Conduct Search All Papers III-C Screening of Papers Relevant Papers III-D Keywording using Title and Abstract Classification Scheme III-E Data Extraction & Mapping Process Systematic Map III-F Now, the individual steps of the 3-step process above will be outlined. Based on the objectives, we first specify the research questions and the review scope in order to formulate search strings for literature extraction. Identify the Scope of our SMS. We already discussed the need for a SMS. We can also clarify the general goal and scope of the study using the PICO (Population, Intervention, Comparison, Outcome) criteria (Kitchenham et al., 2009): PICO concern Explanation Population RQ1: Practical motivation RQ2: Structure and architecture aspects RQ3: Management methods and techniques RQ4: Research challenges and future dimensions Intervention characterise,

internal/external validation; extract data; synthesis Comparison compare by mapping primary studies to characterisation framework Outcome a characterisation framework Define and Evaluate Review Protocol. We developed a protocol based on (Petersen et al., 2008) and on our experience with SLRs (Jamshidi et al., 2013a; Jamshidi et al., 2013b). Conducting the review starts with the study selection and results in extracted data and synthesised information. We specifically focus on orchestration to capture the trend towards distributed container architectures from a research perspective (using orchestration as a broad inclusive term).

B. Definition of Research Questions (Review Scope)

As the next activity, we define the research questions to help shaping the review protocol, see Table I. The main goal of a systematic mapping study is to provide an overview of a research area and to identify the quantity and type of research and results available within it. We can map the frequencies of publication over time to identify trends. A secondary goal is to identify the forums in which research has been published. These goals are reflected in the research questions (RQs).

C. Search for Primary Studies

The selection of search terms is based on (Petersen et al., 2008) and guided by the research questions. The primary studies are typically identified by using search strings on scientific databases or browsing manually through conference proceedings or journals. A common approach to identify the search string is to structure them in PICO terms, which takes into account the research questions. Keywords for the search string can be taken from each aspect of the structure. It is worth noting that, differently from what is suggested by Petersen et al. (Petersen et al., 2008), we do not consider specific outcomes or experimental designs in our study. We avoided this restriction since we wanted a broad overview of the research area as a whole. If we had only considered certain types of studies the overview could have been biased and the

TABLE I: Research Questions (RQ).

RQ Motivation RQ1 (Research Application): Why, in which cloud activities and how have container-based approaches been applied? While containers can be seen as an alternative to VMs at the infrastructure layer, they are also an application packaging mechanism relevant to platform and software-as-a-service. The mechanisms provided need to be organised in a systematic map of the core architecture concerns identified as follows 1) Motivation: what is the motivation for using containers in the cloud (expected benefits)? 2) Technology Stack: How are container-based systems constructed (application/platform)? 3) Management Services, Cloud Settings and Architecture: How are container-based systems developed and managed? Management services are more platform oriented, whereas cloud settings and architecture capture more abstract, higher-level concerns. 4) Technology Space: What concrete cloud/container technologies are used for construction and management? 5) Application Domain: What are containers in cloud actually used

for? RQ2 (Research Distribution): In which sources and when have studies on container technologies in cloud activities been published? The topic of this study is broad (covering cloud, software engineering, distributed systems and operating systems) in terms of communities affected and there should be a number of venues to publish the related studies. This information can help us to identify the leading publication venues where the authors can better disseminate their research results and the trend of the number of published studies in this topic. RQ3 (Maturity): What is the degree of maturity of the field? The research approaches and evaluation methods tell about the maturity of the field, e.g., whether significant empirical studies have been carried out to establish the value in practice or whether the focus is still on investigating technical problems. The relationship of contribution types, e.g., solution proposals vs. experience reports vs. reviews as example can answer maturity questions. RQ4 (Trends): What are the concerns and what is the future research agenda? The aim is to understand and reveal the research gaps and identify future directions. This is a recent concern and the field is still maturing. Questions therefore arise as to what open questions are and what are the remaining challenges for the future. map incomplete. Some sub-topics might be over- or underrepresented for certain study methods. This difference is also reflected in the search string: (cloud* OR PaaS) AND (container*) AND (orchestrate* OR cluster* OR manage*) where ‘*’ matches lexically related terms. Based on the PICO criteria, we chose for 1) Population, here specifically the Technology perspective, the search string (cloud* OR PaaS) AND (container*) AND (orchestrate* OR (cluster* OR manage*)). Initially, further PICO categories were considered, but not applied in the search term: 1) Population - Product perspective: Docker OR Kubernetes OR Diego OR Rocket OR LXC OR ... 2) Intervention: experimental OR empirical OR technical 3) Comparison: n/a 4) Outcome: framework OR theory OR architecture OR design OR language OR use case OR case study The aspects 1 (Product perspective), 2 and 4 were not applied to avoid any incompleteness, but have been considered in the following inclusion/exclusion consideration (part III.D below). Given that this is a recent concern in cloud computing, the corresponding forums are possibly not fully indexed, causing the need for an initial wider, partly manual search. We started with a wider search (i.e., reduced list of terms as suggested above) to establish an overall body of research, which we then narrowed down towards focus and quality. The choice of databases we considered is: IEEE Xplore, ACM Digital Library, Science Direct, ISI Web of Science, SpringerLink, INSPEC, EI Compendex, DBLP. Given the recency of the field and concerns with indexing, Google Scholar played the key role for the initial selection before the inclusion and exclusion stage. Since we used our primary search criteria on title and abstract, this resulted in a high number of irrelevant studies, which were further refined with a secondary search and manual screening - focusing on relevance and resulting then in 46 studies after inclusion/exclusion and quality control.

D. Screening of Papers for Inclusion/Exclusion

The Initial Selection step includes screening of titles and abstracts of potential studies. We use inclusion and exclusion criteria to exclude studies that are not relevant to answer the research questions. The criteria below show that the research questions influence the inclusion and exclusion criteria: Inclusion and Exclusion Criteria

Inclusion The abstract explicitly mentions containers - in general or in the context of cloud computing. From the abstract, we can deduce that the focus of the paper contributes to chosen research focus. Abstract/keywords include key terms and from the abstract it is clear that a contribution towards containers and their management is made.

Exclusion The paper lies outside the container context. Container and their management are not part of the contributions of the paper, the terms are only mentioned in the general introductory sentences of the abstract. Literature not peer-reviewed and only in the form of abstract, blog, presentation are excluded. We started the selection with publications from 2007 with the emergence of the LXC Linux Container technology as a solution for cloud virtualisation and included all relevant publications until the end of 2016. The Final Selection step is a validation scan of the studies, considering methods for cloud container orchestration and tool support and details of the evaluation approach. At the end, 46 studies were selected, listed in Table II.

Qualitative Assessment of Included Studies. For the 46 included studies, we primarily focused on the technical rigor of content presented. We based our assessment on having peerreviewed contributions and selecting those where the content actually matches the title and abstract based initial selection.

Data Extraction and Synthesis. In order to record extracted data from the selected studies, we followed (Petersen et al., 2008) using a structured format based on characterisation dimensions that cover technical, domain-specific categories as well as generic study classification categories.

E. Keywording of Abstracts (Classification Scheme)

Key to our process is keywording to develop the classification scheme and ensuring that the scheme takes the existing studies into account. First, the reviewers involved read abstracts of papers and looked for keywords and concepts that demonstrate the contribution and also the context of each paper. The set of keywords from the different papers considered were then combined into a classification framework

TABLE II: Publications selected - Reference Data.

S1. Elastic Application Container: A Lightweight Approach for Cloud Resource Provisioning. He, Guo, Guo, Wu, Ghanem, Han . Adv Information Netw, Appl Conf. 2012

S2. Distributed Cloud Storage Services with FleCS Containers. Yoon, Ravichandran, Gavrilovska, Schwan . Open Cirrus Summit. 2011

S3. Virtualization vs Containerisation to support PaaS. Dua, Raja, Kakadia . Intl Conf on Cloud Engineering. 2014

S4. Skyport Container-based execution environment management for multi-cloud scientific workflows. Gerlach, Tang, Keegan, Harrison, Wilke,

Bischof, Meyer . Intl Workshop on Data-Intensive Computing in the Cloud. 2014 S5. Containers and Cloud: From LXC to Docker to Kubernetes. Bernstein. IEEE Cloud Computing. 2015 S6. Containerisation and the PaaS Cloud. Pahl. IEEE Cloud Computing. 2015 S7. Virtualization Driven Mashup Container in Cloud Computing PaaS Model. Bheda, Thaker. Intl Conference on Computer Communication and Networks. 2011 S8. vApp: A Standards-based Container for Cloud Providers. Schmidt, Grarup. ACM SIGOPS Operating Systems Review. 2010 S9. Containers and Clusters for Edge Cloud Architectures a Technology Review. Pahl, Lee. Intl Conference on Future Internet of Things and Cloud. 2015 S10. FlexTuner: A Flexible Container-based Tuning System for Cloud Applications. Yu, Zou, Tang, Liu, Teng. Intl Conf on Cloud Engineering. 2015 S11. MultiBox: Lightweight Containers for Multi-Cloud Deployments. Hadley, Elkhatib, Blair, Roedig. EGC Workshop. 2015 S12. Self-Adaptive Containers: Interoperability Extensions and Cloud Integration. Huang, Knottenbelt. UIC-ATC-SCALCOM '14 Associated Workshops. 2014 S13. Flexible Network Address Mapping for Container-based Clouds. Kim, Lee, Ben-Ami, Nam, Janak, Schulzrinne . Conf on Network Softwarization. 2015 S14. Cross-Platform and Cloud-Based Access to Multiple Particle Accelerator Codes Via Application Containers. Bruhwiler, Nagler, Webb, Andonian, Harrison, Seung, Moeller . Particle Accelerator Conf. 2015 S15. About Automatic Benchmarking of IaaS Cloud Service Providers for a World of Container Clusters. Kratzke, Quint. Journal of Cloud Computing Research. 2015 S16. A Container-based Elastic Cloud Architecture for Real-Time Full-Motion Video (FMV) Target Tracking. Wu, Chen, Blasch, Liu, Chen, Shen . IEEE Applied Imagery Pattern Recognition Workshop. 2014 S17. Container-based orchestration in cloud: state of the art and challenges. Tosatto, Ruiiu, Attanasio. Intl Conf on Complex, Intelligent, Software Intensive Systems. 2015 S18. A REST Service Framework for Fine-Grained Resource Management in Container-Based Cloud. Li, Tang, Chou. IEEE Intl Conference on Cloud Computing. 2015 S19. A Performance Isolation Analysis of Disk-intensive Workloads on Container-based Clouds. Xavier, De Oliveira, Rossi, Dos Passos, Matteussi, De Rose . Intl Conf on Parallel, Distributed, and Network-Based Processing. 2015 S20. The SPD approach to deploy service-based applications in the cloud. Yanguai , Tata. Concurrency and Computation: Practice and Experience. 2014 S21. Large-scale cluster management at Google with Borg. Verma, PedroL, Korupolu, Oppenheimer, Tune, Wilkes. European Conference on Computer Systems . 2015 S22. Harbormaster: Policy Enforcement for Containers. Zhang, Marino, Efstathopoulos . Intl Conference on Cloud Computing Technology and Science. 2015 S23. Integrating Containers into Workflows: A Case Study Using Makeflow, Work Queue, and Docker. Zheng, Thain. Intl Workshop Virtualization Techn in Distr Comp. 2015 S24. Performance Evaluation of Microservices Architectures using Containers. Amaral, Polo, Carrera, Mohamed, Unuvar, Steinder . Intl Symp Network Comp & Appl. 2015 S25. Umbrella: A Portable Environment Creator for Reproducible Computing on Clusters, Clouds, and Grids. Meng, Thain.

Workshop Virtualization Techn in Distr Comp. 2015 S26. A Lightweight Virtualization Cluster Reference Architecture Derived from Open Source PaaS Platforms. Kratzke. Open J Mob Comput Cloud Comput. 2014 S27. Docker. Anderson. IEEE Software. 2015 S28. Concerning Containers' Connections: on Docker Networking. Kereki. Linux Journal. 2015 S29. Exploring Containers for Scientific Computing. Gomes, Pina, Borges, Martins, Dias, Gomes, Manuel . Iberian Grid Infrastructure Conference. 2014 S30. Hypervisors vs Lightweight Virtualization: a Performance Comparison. Morabito, Kjallman, Komu . Intl Conf on Cloud Engineering. 2015 S31. Dynamic Tailoring and Cloud-based Deployment of Containerized Service Middleware. Saez, Andrikopoulos, Sanchez, Leymann, Wettinger . Intl Conf on Cloud Comp. 2015 S32. Performance evaluation of containers for HPC. Ruiz, Jeanvoine, Nussbaum . European Conference on Parallel Processing. 2015 S33. The Role of Container Technology in Reproducible Computer Systems Research. Jimenez, Maltzahn, Moody, Mohror, Lofstead, Arpaci-Dusseau, Arpaci-Dusseau . Intl Conf on Cloud Engineering. 2015 S34. A DevOps Approach to Integration of Software Components in an EU Research Project. Stillwell, Coutinho. Intl Workshop on Quality-Aware DevOps. 2015 S35. Container-based Operating System Virtualization: A Scalable, High-performance Alternative to Hypervisors. Soltesz, Ptzl, Fiuczynski, Bavier, Peterson . OS Review. 2007 S36. Performance Evaluation of Container-based Virtualization for High Performance Computing Environments. Xavier, Neves, Rossi, Ferreto, Lange, De Rose . Intl Conf on Parallel, Distributed, and Network-Based Processing. 2013 S37. A Discrete-Time Feedback Controller for Containerized Cloud Applications. Baresi, Guinea, Leva, Quattrocchi . Symposium on Foundations of Software Engineering. 2016 S38. MicroCloud: A Container-based Solution for Efficient Resource Management in the Cloud. Baresi, Guinea, Quattrocchi. Smart Cloud Conf. 2016 S39. Toward a Standard Interface for Cloud Providers. Fu, Liu, Chu, Hu . IEEE Internet Computing. 2016 S40. Container and Microservice Driven Design for Cloud Infrastructure DevOps. Kang, Le, Tao. Cloud Engineering Conf (IC2E). 2016 S41. Flexible Container-Based Computing Platform on Cloud for Scientific Workflows. Liu, Aida, Yokoyama. Cloud Computing Conf. 2016 S42. Auto-tuning Performance of MPI Parallel Programs Using Resource Management in Container-based Virtual Cloud. Ma, Wang, Tak, Wang, Tang. Cloud Comp Conf. 2016 S43. Design of an IoT Cloud System for Container Virtualization on Smart Objects. Puliafito, Villari. Advances in Service-Oriented and Cloud. 2016 S44. A Container-based Edge Cloud PaaS Architecture based on Raspberry Pi Clusters. Pahl, Helmer, Miori, Sanin, Lee . Symp FI Things and Cloud. 2016 S45. Container-Based Cloud Virtual Machine Benchmarking. Varghese, Subba, Thai. Cloud Engineering Conf (IC2E). 2016 S46. Container based Video Surveillance Cloud Service with Fine-Grained Resource Provisioning. Zhang, Ma, Fu, Yang, Jiang . Cloud Computing Conf. 2016 to develop a high level understanding of the subject, which is representative of the underlying population. We combined a number of top-level categories, both technical domain-

specific concerns as well as those on the organisation of the research. The generic categories are adopted from the literature, e.g., the contribution type, as in the table below. Category Description of Research Contribution Validation Research Techniques investigated are novel and have not yet been implemented in practice. Techniques used are for example experiments, i.e., work done in the lab. Evaluation Research Techniques are implemented in practice and an evaluation of the technique is conducted. It is shown how the technique is implemented (solution implementation) and what are the benefits and drawbacks of the implementation (implementation evaluation). Solution Proposal A solution for a problem is proposed, the solution can be either novel or a significant extension of an existing technique. The potential benefits and the applicability of the solution is shown by a small example or a good line of argumentation. Philosophical Papers These papers sketch a new way of looking at existing things by structuring the field in form of a taxonomy or conceptual framework. Opinion Papers These papers express the personal opinion of somebody whether a certain technique is good or bad, or how things should be done. They do not rely on related work and research methodologies. Experience Papers Experience papers explain on what and how something has been done in practice. It has to be the personal experience of the author. The remaining categories are specific to container and cloud technology and aim to answer the research questions (specifically RQ1.1 to RQ1.5) in a targeted way.

- Generic Concerns: Research Contribution, Evaluation Method, Forum, Community, Domain, and Motivation for Technology Adoption.
- Technology-Specific Concerns: Technology Stack, Management Services, Architecture Setting, and Tools/Platforms/Technology.

This technical perspective is highlighted in Fig. 3 where these technical concerns are visualised. The four main concerns Technology Stack, Architecture, Management and Tools/Platforms/Technology are aligned with the research questions RQ1.2 to RQ1.4, whereby we split RQ1.3 into a conceptual, methodology-oriented Architecture aspect and a platform-oriented Management aspect that covers the available support services. The concerns are centred around the Technology Stack which is about the container technology itself and its internal mechanisms, whereas the surrounding three

others are about the cloud environment in which containers are used and supported. The categorisations within the top-level concerns have emerged from the keyword extraction and have been aligned with the 4 top-level RQ1-oriented categories.

Fig. 3: Categories of Cloud Container Orchestration Concerns.

A first-round keyword extraction has been used to validate the categories, i.e., that concrete terms extracted from the studies occur as instances of the categorisation scheme. F . Data Extraction and Mapping of Studies (Systematic Map) When the classification scheme is defined, the actual data extraction is the process of categorising the relevant studies into the scheme¹ The classification scheme has evolved during the data extraction (adding new categories and merging or splitting existing ones), based on further input processing and feedback received from

independent experts. Data during this process has been gathered in a spreadsheet table to document the data extraction process. From the final table, the frequencies of publications in each category can be calculated. This allows to see which categories have been emphasized in past research and thus to identify gaps and possibilities for future research. We combined two maps - on generic research concerns and on domain-specific technology concerns. For the two maps, we use different ways of presenting and analysing the results. Common generic map targets are: Trends (by year), Forums, and Frequency (by topic). The technology map targets construction aspects (Technology Stack) and management aspects (Architecture and Management Services or Tools, Platforms and Technologies). The mapping is here from an enriched classification framework to the current research coverage (as shown by the selected papers). The map is illustrated in Section V using statistics in form of tables and pie charts, showing the frequencies of publications in each category.

IV. A CLASSIFICATION FRAMEWORK FOR CLOUD

CONTAINER TECHNOLOGIES We first introduce a reference model for an architecturecentric classification of cloud container technologies that helps to demonstrate current research at a conceptual level and identify trends and research directions. 1Our actual extracted data is presented at [http://www.inf.unibz.it/~cpahl/CCT/Cloud Container Technology-a State-of-the-Art Review.htm](http://www.inf.unibz.it/~cpahl/CCT/Cloud%20Container%20Technology-a%20State-of-the-Art%20Review.htm).

TABLE III: Classification Framework.

We propose this classification framework, see Table III, to categorise the primary studies. This framework uses common terms from software engineering methods with methodological support, architecture, tool support and applications. • The top-level headings (column 1) were already discussed in their alignment with the research questions and illustrated in Fig. 3. • The subheadings (columns 2 to 4, if applicable) were identified, taking into account the research questions, after a first-round scan of selected studies in order to ensure the validity of the framework. These align with categorisations found in technology reviews (Pahl, 2015) for the technology stack, management services, cloud architecture settings and the cloud technologies. • The concrete terms (last column on the right-hand side of each row) are the terms extracted from the study. We organised them manually into the subheadings above.

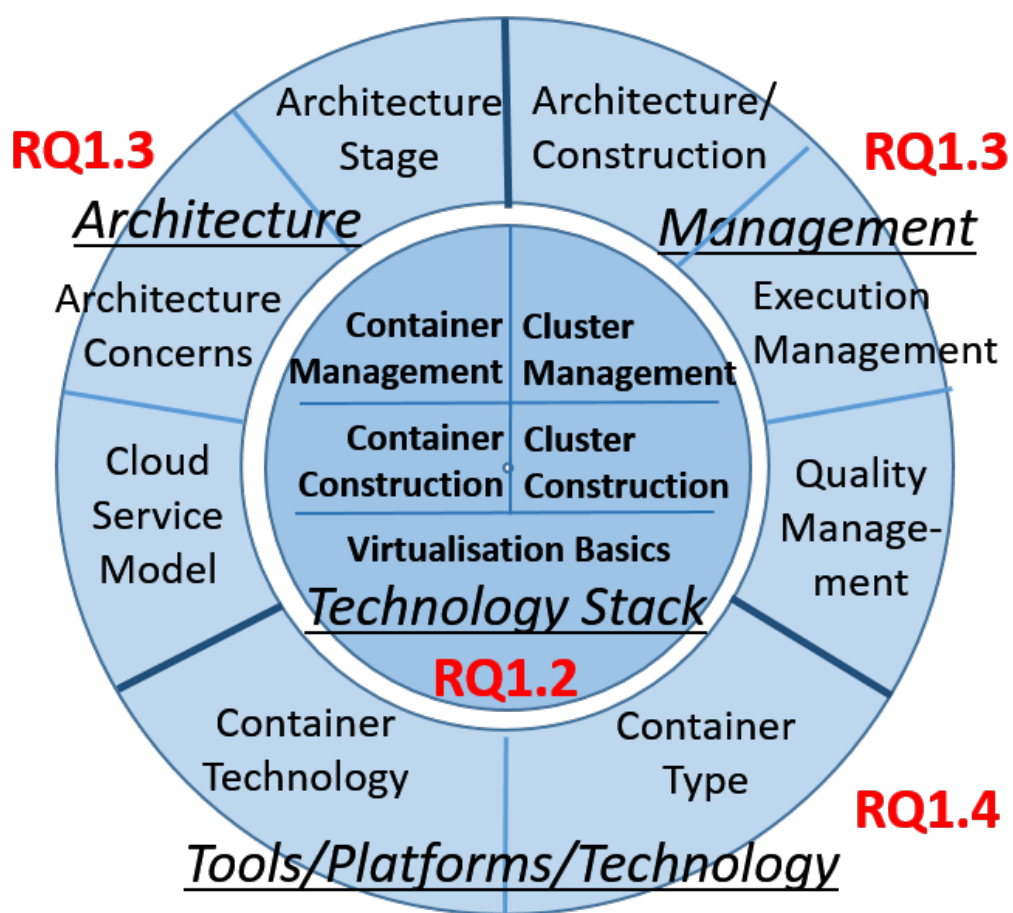


图 3：云容器编排关注点分类。

Contribution Type	Solution proposal (Architecture, Framework, Methodology, Library), Evaluation research, Validation research, Experience report, Review (SOTA review, Technology, SLR)			
Evaluation Method	Case study, Mathematical proof, Experience report, Example application, Controlled experiment			
Forum	Journal, Magazine, Conference/Workshop			
Technology Stack	Virtualisation Basics	virtualisation/VM, isolation, hypervisor, OS, control group, namespace		
	Container Construction	Activity construction, application packaging/assembly, provisioning, image building; Container type mashup, micro, meta		
	Container Management	management, communication, application execution		
	Cluster Construction	definition, construction		
	Cluster Management	management, communication, execution		
Management Services	Architecture/Construction	selection, integration, arch construction/composition, topology, microservice architecture, interoperability/heterogeneity, migration/cloudification, fault handling, adaptivity		
	Execution Management	orchestration, configuration, installation, provisioning, start-up, delivery, load management, scheduling, network address mapping, network management (routing, proxy)		
	Quality/SLA Management	Concern/Type:	Moni-torable	performance, elasticity, security, workload, size/volume, resource utilisation, compliance, startup time, reliability, memory use
			non-moni-torable:	resilience, portability, interoperability
			testable:	integration, scalability, configurability
		Activity	tuning, self-benchmarking, self-adaptivity	
Architecture Setting	Deployment Stage	requirement, design, deployment, migration, DevOps		
	Architecture Concern	flexibility, modularity, (self-)adaptiveness, integration, interoperability, quality		
	Cloud Setting	single/private, multi/cross/hybrid, edge/fog, hosted, IoT-cloud, clustered; IaaS, PaaS, SaaS, XaaS		
Tools / Platforms / Technology	Technology	Docker, Kubernetes, Diego, Rocket, CoreOS, LXC, OpenVZ, STXXL, MCSTL, Intel TBB, Xen, OVF, EAC		
	Container Type	Container, Cluster manager, Unikernel		
Domain	Computing	big data, video, workflow management, HPC, Web application, disk-intensive workloads, resource virtualisation		
	Non-Computing	health, physics/science, media, education		
Community	Distributed Systems, Cloud and Big Data, Software Engineering, Autonomic Computing, Network/OS, Application			

The framework has initially undergone several iterations between the authors of this study and have then been validated by five external experts at three organisations in three countries. This has resulted in some corrections and amendments of the first version based on the extraction.

V. RESULTS AND VISUALISATION

Table IV overviews all selected studies based on the most important categories. The table positions each study using the classification framework, but also to obtain a first overview of the coverage of all studies together. For the aspects ‘technology stack’ and ‘management’, we only capture the focus as in these categories multiple occurrences of terms were possible (for which there is not sufficient space in the table). For the architecture setting and the research aspects, we listed all terms extracted. It is worth noting that review papers (a) tend to cover the technology stack more widely and solution papers tend to focus more on specific layers, (b) there is more work on deployment and management services than design and architecture concerns, (c) there is an equal spread between IaaS and PaaS focus, and (d) that mainly experiments and case studies are used for evaluation. We categorise the selected papers in terms of concerns such as publication format, forum and technical contribution. We also present the key terms extracted from the studies. The results are discussed and the validity of the results and their impact for future research is addressed. The two maps generic and technology-specific - are discussed separately in this section. We hereafter present the results of our study in more detail (generic and domain-specific maps are presented separately). We also discuss the validity of our results and their potential impact for future research.

A. Overview of Primary Studies - Generic Attributes

In order to examine the current state of research on cloud container orchestration, the following questions apply: • When did research on container technologies become active in the cloud computing community? • What are the fora in which work on cloud container technologies has been published? On which cloud-related communities does the focus lie? • How is cloud container technologies research reported and what is the maturity level of the research in this field within cloud computing? 1) Temporal overview of studies: With only a few studies on container orchestration in the early years after the LXC introduction, there has been a dramatic increase since the emergence of tools like Docker, signalling a significant concern, see Fig. 4. The 2007 paper ‘Container-based operating system virtualisation: a scalable, high-performance alternative to hypervisors’ is however the most cited one. It can be considered a pioneer in looking at containers as a solution for cloud computing, then confirmed by the interests raised in that paper three years later. This paper brings container technology into the cloud research domain. However, there was not much open source technology in existence that researchers could consider. As a consequence, only after the recent introduction

Fig. 4: Study Distribution by Year. At the time of writing,

the amount of (already indexed) paper published in 2016 is lower than that of 2015. This is most probably because of the delays induced by the online publication of proceedings, by the review processes in journals, and by online indexing leaving the 2016 list of papers inevitably incomplete. of Docker containers, researcher picked up this direction in research more strongly. 2) Publication fora/communities and formats: We distinguish communities and formats. Fora/Communities: We have categorised the publication fora into the computing fields as follows (Fig. 5): Software Engineering, Autonomic Computing, Distributed Systems, Cloud and Big Data, Network/OS, Application. Publications did occur mainly in the now established cloud computing field, as containers can play an important role in cloud (especially PaaS) virtualisation. Other areas include the networks, distributed systems and software engineering communities. We can note that there is not much work in terms of software development for containers and potentials of auto-scaling with containers. Formats: Regarding the sources, we recognise and distinguish the following peer-reviewed publication formats: journals, magazines, conferences and workshops, and books, see

Fig. 6. At this formative stage, we can see mainly workshops

and conferences, with magazines and journals often being survey papers. Researchers go mainly for conferences/workshops, as they can provide feedback and publish results in a shortest time, which is of high value in this fast developing area.

Fig. 5: Study Distribution by Community.

3) Research Methods: Contribution type: In Fig. 7, the primary studies are summarised according to their contribution type. Solution proposals dominate, with little on validation and evaluation at this (formative) stage. Reviews are technology reviews rather than SLRs. More comparisons among existing solutions are needed (for which our classification scheme can be a compass to organise such solutions).

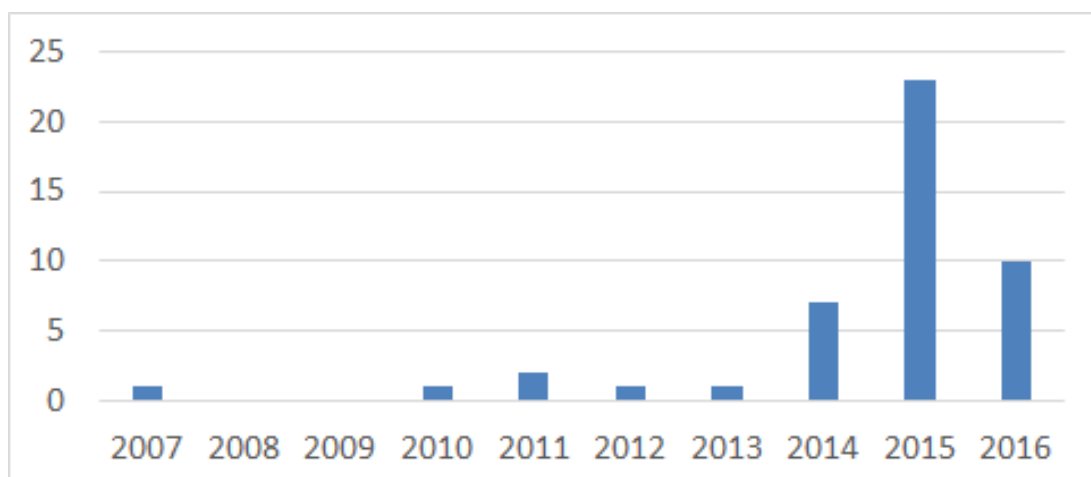


图 4: 研究按年份分布。

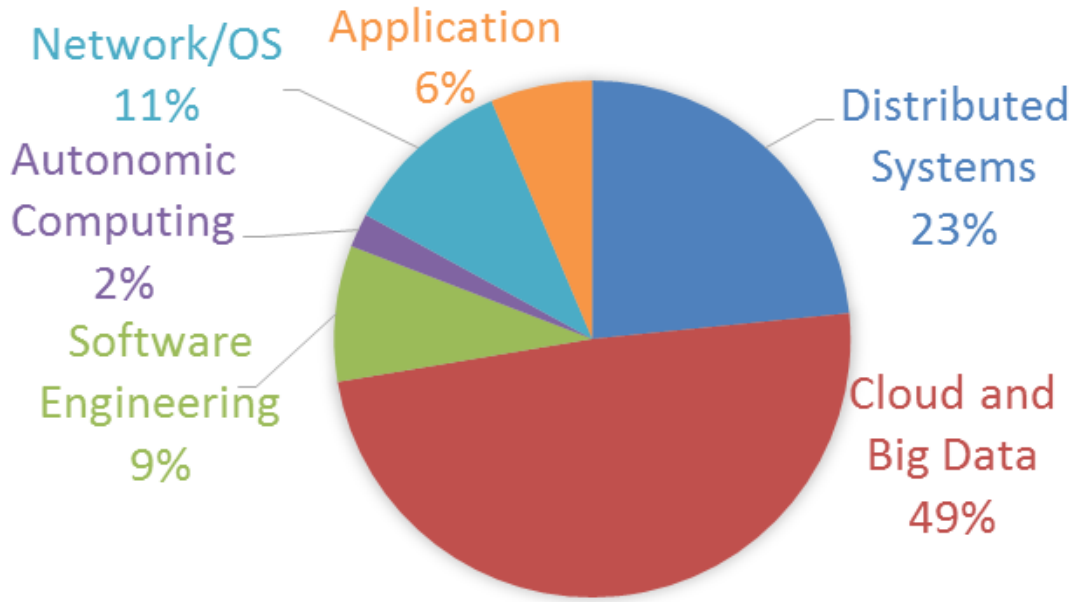


图 5: 研究按领域社区分布。

TABLE IV: Study Comparison - Selected Categories.

Fig. 6: Study Distribution by Publication Format.

Fig. 7: Study Distribution by Contribution Type.

Evaluation Method: Given the relative immaturity of the domain, the evaluations lack detailed experience reports and proofs, while sample implementations as experience reports and some controlled experiments have been reported - which matches the solution proposals as the contribution types. Fig. 8 looks at the evaluation methods used. Figs. 10 to 16 complement this by looking more specifically at the technical contribution itself from different perspectives. No mathematical/theoretical foundations are behind the solutions we analysed (as all of them are technological solutions, rather than theoretical approaches). This provides a pointer for further investigation/future work on providing container orchestration with theoretical foundations.

Fig. 8: Study Distribution by Evaluation Method.

Summary: We can observe a relatively young field, gaining more and more interest over the last years, especially (for

	Technology Stack					Management Services			Architecture Setting				Research Methodology	
	Virtualisation Basics	Container Construction	Container Management	Cluster Construction	Cluster Management	Architect/Construct	Execution Management	Quality Management		Deployment Stage/Context	Architecture Concern	Cloud setting	Contribution type	Evaluation method
								monitorable	non-monitorable					
1	x	x	x	x	x	x	x	x		deployment	flexibility, modularity	IaaS	Solution	Experiment
2	x	x					x			deployment, migration	flexibility, integration		Solution	Experiment
3	x	x	x				x	x		design, deployment	flexibility	PaaS	Review	Experiment
4	x						x			deployment	flexibility, adaptive		Solution	Example
5	x									deployment	interoperability	PaaS	Review	
6	x	x	x	x	x	x	x	x		deployment, migration	flexibility, modularity	PaaS	Review	
7	x					x	x		x	deployment	flexibility, adaptive	PaaS	Solution	
8	x									deployment	integration	IaaS	Solution	Example
9	x	x	x	x	x	x	x			deployment, DevOps	modularity	PaaS	Review	
10		x				x	x	x		deployment	flexibility	IaaS	Solution	Case study
11		x				x	x	x		migration	flexibility	IaaS	Solution	Case study
12		x				x	x		x	deployment	adaptive	IaaS	Solution	Case study
13	x	x					x			deployment	integration	IaaS, PaaS	Solution	Experiment
14	x	x				x				migration, deployment	flexibility	PaaS	Solution	Case study
15	x					x	x	x		design	quality	IaaS	Solution	Case study
16	x	x	x				x	x		deployment	flexibility	IaaS	Experience	Case study, Experiment
17	x	x	x				x	x	x	deployment	flexibility, quality	IaaS, SaaS	Review	
18	x		x			x	x	x	x	deployment	integration	PaaS	Solution	Experiment
19	x	x								deployment	quality	IaaS, PaaS	Review	Experiment
20		x	x			x	x	x		requirement, design, deployment	flexibility	PaaS	Solution	Case study
21	x	x				x	x	x		deployment	flexibility	PaaS, SaaS	Solution	Experiment
22	x	x				x				requirement, design, deployment	flexibility, quality	PaaS	Solution	Experiment
23	x	x	x				x	x		design, deployment	integration	PaaS, SaaS	Solution	Case study, Example, Experiment
24	x	x				x	x	x		design, deployment	flexibility, modular., interop.	PaaS	Evaluation	Experiment
25	x					x	x		x	deployment, migration	interoperability, integration	IaaS, PaaS	Solution	Case study, Experiment
26						x	x	x		requirements, design	modularity	IaaS, PaaS	Solution	
27	x	x	x							design, deployment	flexibility, quality	PaaS	Solution	
28	x	x	x				x		x	deployment	integration	IaaS	Experience	Example
29	x	x	x					x	x	design, deployment	integration	IaaS	Solution	
30	x	x	x				x	x	x	deployment	modularity	IaaS	Solution	Experiment
31	x	x	x	x	x	x	x	x		design, deployment	quality	PaaS	Solution, Evaluation	Experiment
32	x	x	x			x	x	x	x	deployment	integration, quality	IaaS	Evaluation	Experiment
33		x	x				x			deployment	integration, quality	IaaS	Solution	Case study
34	x	x					x	x	x	design, deployment, DevOps	integration	IaaS	Solution	Case study
35	x	x	x				x	x		deployment	flexibility, integration, quality	IaaS	Solution, Review	Experiment
36	x					x		x		quality	quality	IaaS	Evaluation	Experiment
37	x	x	x			x	x	x	x	deployment	adaptive	IaaS	Solution	Experiment
38	x	x	x			x	x	x		deployment	adaptive	IaaS	Solution	Experiment
39	x	x				x	x	x		deployment	quality	IaaS, PaaS	Review	
40	x	x	x			x	x	x	x	DevOps	flexibility, modularity, adaptive	PaaS	Solution, Review	Experiment
41		x				x	x			deployment	quality	IaaS	Solution, Evaluation	Experiment
42	x	x	x			x	x	x		deployment	quality	IaaS	Solution	Experiment
43	x		x	x	x		x	x		design, deployment	flexibility, integration, interop	PaaS	Solution	Example
44	x	x	x	x	x	x	x	x		deployment	flexibility, integration	IaaS, PaaS	Solution	Experiment
45	x						x	x		deployment	quality	IaaS	Solution, Evaluation	Experiment, Case Study
46			x				x	x		deployment	flexibility, quality	IaaS	Solution, Validation	Example, Experiment

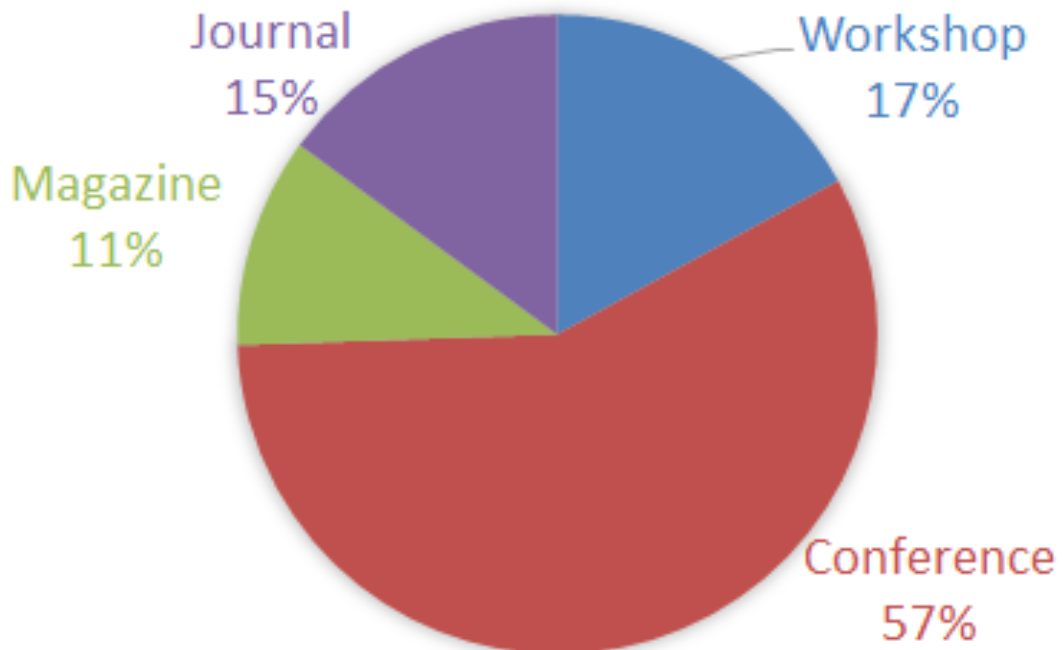


图 6：研究按发表形式分布。

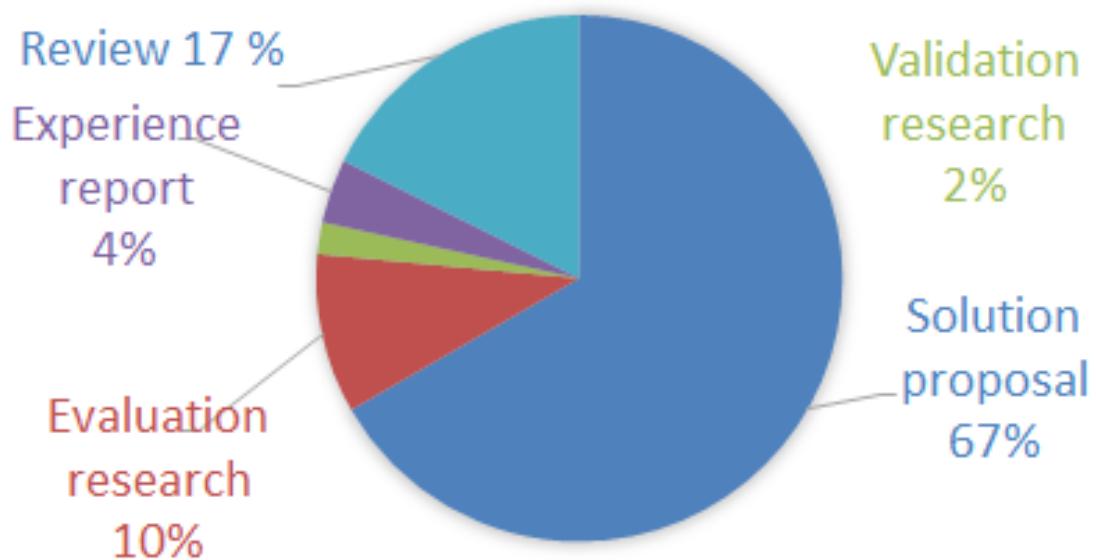


图 7: 研究按贡献类型分布。

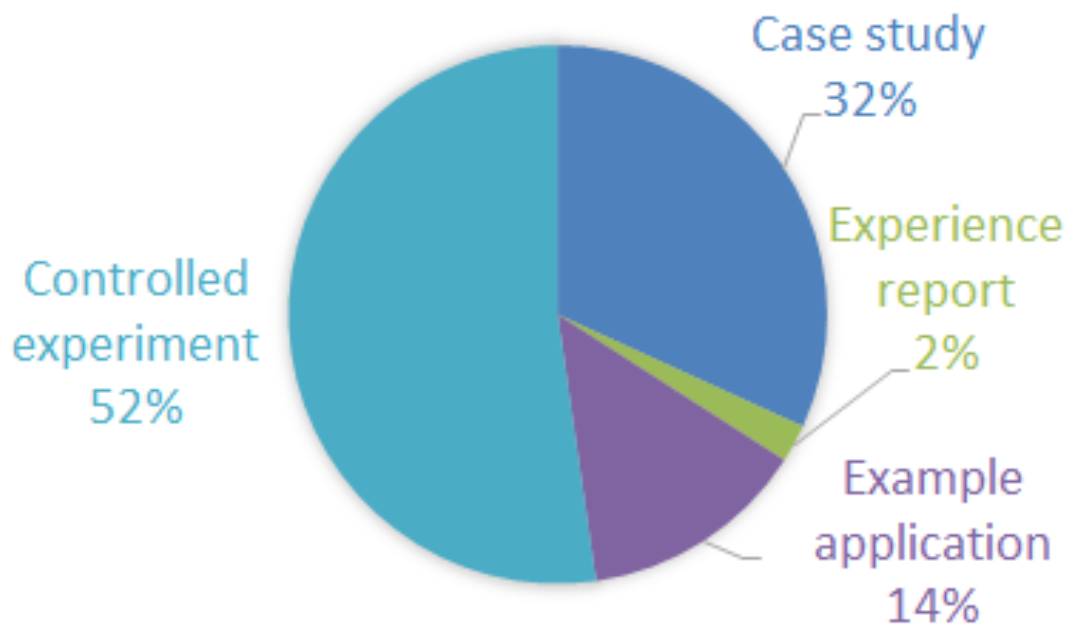


图 8: 研究按评估方法分布。

conferences) in the cloud domain..

B. Overview of Primary Studies - Domain-Specific Attributes

Now we focus on the technical aspects of the classification. We will visualise the data again in pie-charts (%-based) and provide the respective total numbers in tables. We also provide some additional groupings of terms to clarify further the priorities set by research 2. 1) Motivation: Ease of deployment and lightweight resource management are seen as the key benefits. Largescale and automatic deployment are also key motivations, see

Fig. 9. Hence, ease of deployment/management of large-scale

heterogenous applications are the main drivers (even before lightweight resource

management).

Fig. 9: Study Distribution by Motivation.

Fig. 10: Study Distribution by Technology Stack Layer.

2) Technology Stack: The technology stack covers the layered construction of virtual resources, which links platform to middleware and orchestration aspects within PaaS, Fig. 10. The focus is on container construction and management (more management than construction), with a good amount of work on virtualisation basics, but also some recent work on clusters. Technology Stack - Virtualisation Basics: The results here are quite obvious (containerisation is a form of virtualisation), but there is some notable interest in isolation. This is mainly geared towards addressing the security concerns that are due to the container-based virtualisation (where containers are essentially guest processes on a shared operating system, and malicious users could cause more security issues with respect to virtual machines managed by hypervisors). However, according to [S3], containers promise the same level of isolation and security as VMs. No dependence on hardware emulation provides performance benefits over full virtualisation, but restricts the number of supported operating systems. As this is not often required by PaaS providers, containers are suited 2In the tables, we summarised for each subcategory a ranked list of term occurrences, highlighting the more relevant concepts, such as an interest in isolation properties. Note, that the terms in the categories can occur multiple times across different studies (with at most one occurrence counted per study). for providing low-overhead isolation. Container features like cgroups can limit device access for containers from a security perspective, but filesystem, network and memory isolation need more attention. Technology Stack - container construction is more about (functional) composition. The focus is on the actual imagebased construction (assembly) process. Technology Stack - container management is on the other hand operations and management oriented. Core execution and wider (quality) management are equally considered; there is some interest in communications of multi-container applications. Technology Stack - cluster construction and management as distributed architectures reflect the previous two observations for containers. Virtualisation Basics Count virtualisation/VM 20 isolation 14 OS 5 control group 5 namespace 5 hypervisor 4 Container Construction Count construction 14 provisioning 13 application packaging 10 image building 5 Container Management Count management 22 application execution 18 communication 7 Cluster Construction Count construction 8 definition 2 Cluster Management Count management 12 execution 5 communication 4 There is a considerable interest in the core virtualisation aspects of containers, looking into the optimisation of performance for storage [S19] and network aspects [S13]. Other directions beyond orchestration are security concerns (Manu et al., 2016) such as isolation.

Fig. 11: Study Distribution by Management Service

3) Management Services: The Management Services address both development and operations/management perspectives by looking at the support services that are provided by a container platform. For example, software engineering activities can be supported by middleware/platform services, software architecture and quality assurance, performance engineering and distributed systems concerns. These are categorised into architecture/construction, execution management and quality management, see Fig. 11. Management Services - Architecture/Construction : The main focus is on classical single-system architecture concerns, but also on system integration (architecture and distribution/topology management) and some quality and change

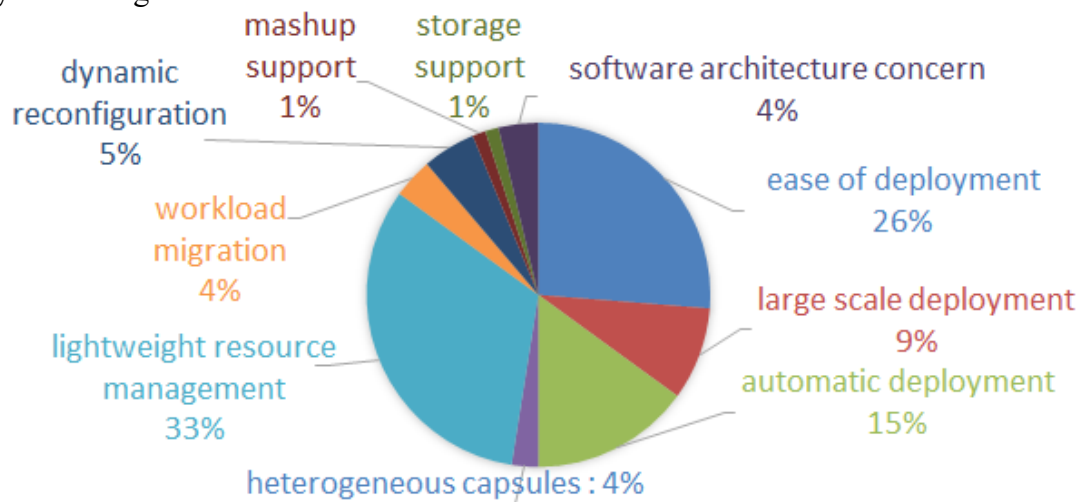


图 9：研究按采用动机分布。

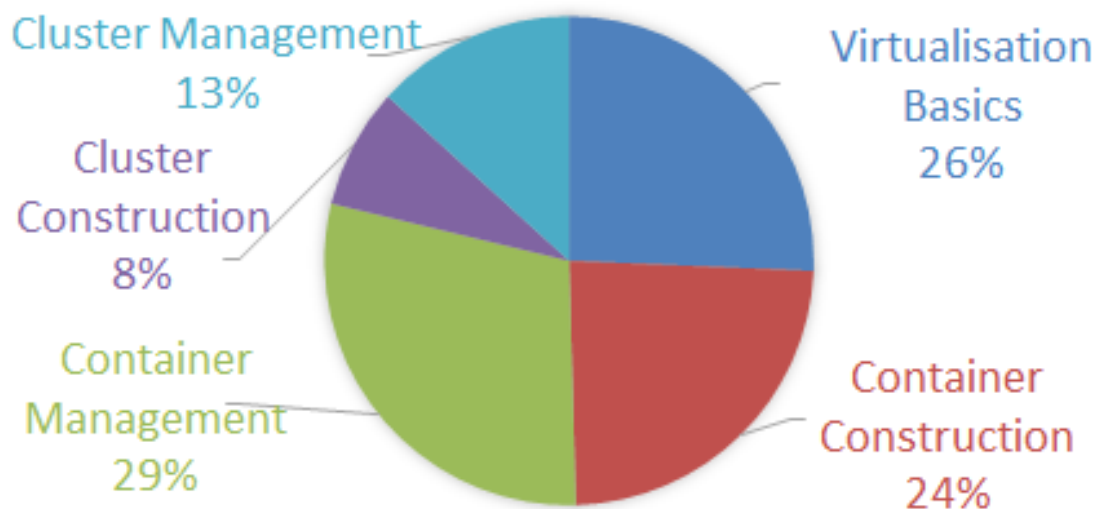


图 10：研究按技术栈层次分布。

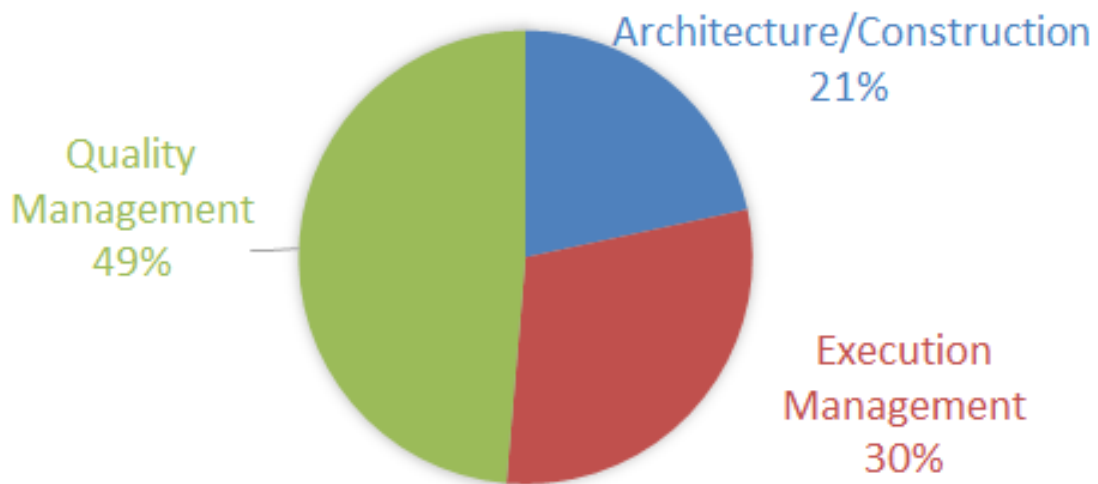


图 11: 研究按管理服务分布。

management can be noted: Architecture/Construction Count construction/composition 10 interoperability/heterogeneity 7 topology (cluster/distribution) 4 adaptivity 4 microservice architecture 4 integration 3 selection 2 cloudification/migration 2 fault handling 2 cloud-native 1 A grouping shows that construction of application containers and their integration are the top concerns: Arch/Contr Group Terms Count Selection & Construction selection + construction/composition + microservices Integration integration + interoperability/heterogeneity 9 Quality Management adaptation + fault handling 6 Distribution Management topology 4 Evolution cloud / migration 2 Integration in environments with interoperability challenges due to heterogeneity is the second strongest group. The benefit of container technology to aid the orchestration of applications in distributed topologies is highlighted in [S9], the emerging trend towards edge cloud computing and its orchestration needs are also noted. Cluster management is instead covered on [S21], [S25] and [S26]. Management Services - Execution Management: the following concerns have been recorded: Execution Management Count orchestration 12 provisioning 10 start-up 7 load management 7 configuration 4 installation 4 scheduling 4 network address mapping 4 delivery 3 network management (routing, proxy) 3 This can again be grouped. There is a balance between set up/preparation, core execution management, wider (quality) management and, almost as much, enabling network/infrastructure technology. Exec Mgmt Group Terms Count Advanced Management load management + scheduling + orchestration 23 Preparation configuration + start-up + installation 15 Core Execution delivery + provisioning 13 Network/Infrastructure network management + address mapping 7 While performance and scalability are accepted IaaS concerns, managing performance in container-based application architectures is also a PaaS concern [S24,S32]. Performance is central in all aspects here, from load management to configuration to provisioning and efficient network management. Management Services - Quality Management results are below: The three categories monitorable /nonmonitorable/testable are split 56 /8/6, respectively. Monitorable and testable quality aspects

are in the majority and this can be attributed to the large number of experimental studies in the systems community. Quality Category Terms Count monitorable performance 23 resource utilisation 13 startup time 10 elasticity 10 security 7 reliability 4 memory use 4 workload 3 size/volume 3 compliance 1 non-monitorable portability 5 interoperability 5 resilience 2 testable scalability 6 configurability 2 integration 1

There is a balance between infrastructure quality parameters, (external/given/provided) service-level objectives (SLO) and some parameters describing the adaptation mechanism inbetween (e.g., elastic), as the grouping below shows:

Quality Group	Terms	Count
SLA	Parameter performance, compliance, security, reliability	36
Infrastructure	Parameter workload, resource utilisation, startup, memory use, size/volume	
System/Mgmt	Parameter elasticity	10

The fact that the aim is gain in performance (less overhead) through containers as virtualisation mechanisms is noteworthy [S13,S19,S30,S35,S36,S39]. [S1] reports that containers are more resource efficient and more scalable due to the significantly lower RAM consumption (29.4 times smaller than VM). High-performance computing (HPC), where both isolation and performance is needed, benefits since containerbased virtualisation provides less overhead than hypervisorbased environments and isolation concerns are less prevalent in HPC due to limited resource sharing needs [S36,S42]. There is interest both in both SLAs and infrastructures. Concerning SLAs the predominant parameter is performance, while for infrastructures the predominant parameters are resource utilisation and portability. What emerges here is a stakeholder dimension: SLA concerns are important for the consumers/users of the application, infrastructure concerns are related to the providers and the system/management category addresses an internal perspective.

Fig. 12: Study Distribution by Stage

Fig. 13: Study Distribution by Architecture Concerns

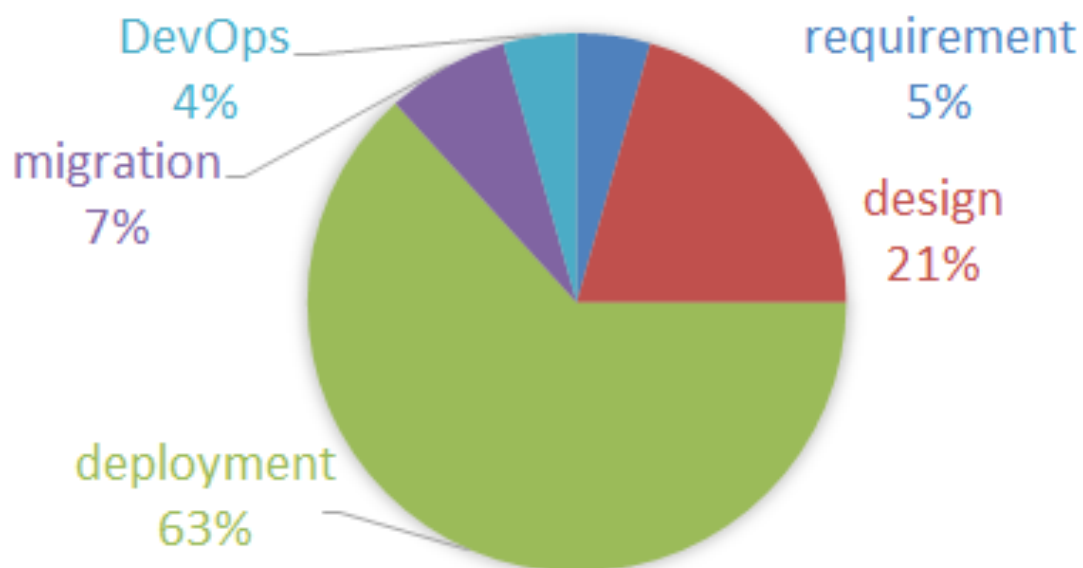


图 12: 研究按开发阶段分布。

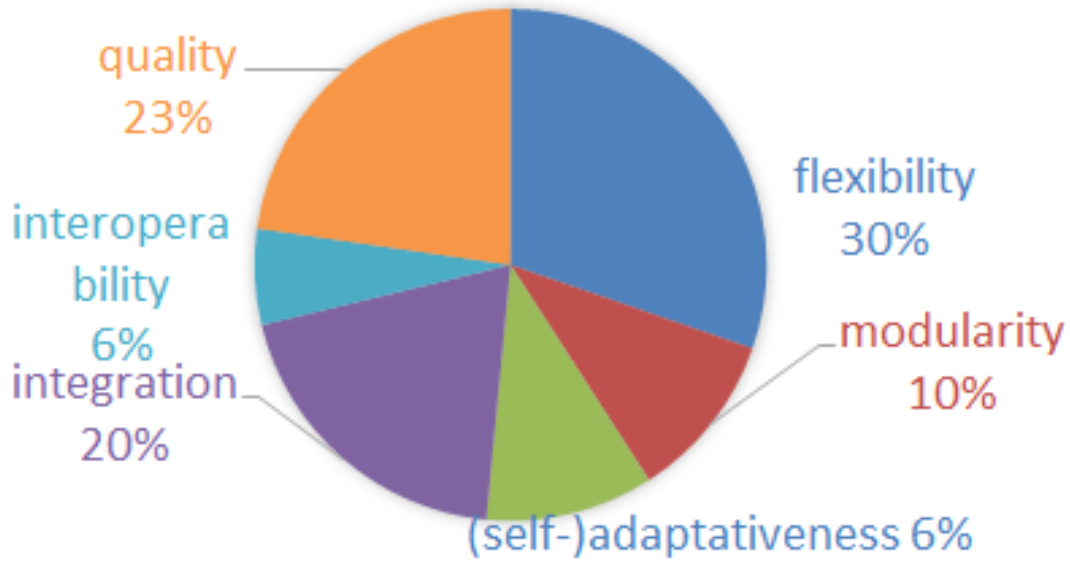


图 13: 研究按架构关注点分布。

4) Architecture Setting: Wider software engineering concerns are also covered by our classification scheme, which extends the Architecture/Construction and Management Services, but at a higher level. • ‘Stage’ refers to the development stage, Fig. 12, which can be summarised into three groups of concerns: Group of Stages/Activities Count requirements + design 17 deployment 43 migration + DevOps 8 The concern is mainly deployment, with less design stage activities and only a few cross-cutting concerns (such as DevOps). This confirms other observations above. The operational concerns of deployment and runtime monitoring and development aspects like testing would benefit from a tighter integration. For instance, [S34] shows that the combination of automated testing and deployment improves the speed and efficiency. Continuous deployment and automation of activities in a DevOpsstyle can be the solution. • Architecture Concerns, see Fig. 13: On a term-by-term basis, ‘flexibility’ emerges as a key benefit for clouds and containers, but also that applications have to be integrated and run under quality requirements. Fig. 13 is detailed in the table below. We can observe an equal balance between construction and management (change and adaptivity) towards DevOps and continuous development [S20,S40]. Other qualities are less relevant. Architecture Group Terms Count Construction modularity + integration + interoperability 24 Change flexibility + self-adaptation 27 Other qualities [remaining qualities] 15 5) Cloud and Container Technology Space: • Cloud Delivery Model : we observe a mix of IaaS and PaaS. The data we collected clearly shows that containerisation is useful at the level of IaaS and PaaS, with both IaaS and PaaS almost equally distributed. A few more contributions target IaaS from a virtualisation and orchestration perspective, rather than PaaS with its application packaging focus. IaaS is often present in the analysed papers. Many of them also address virtual machines and compare how containers perform due to having a lighter virtualisation approach (Felter et al., 2015;

Dupont et al., 2016). VMs are the traditional model for IaaS, but containers are similar as imagebased machines, but also oriented towards PaaS through application packaging aspects, typically running on top of a virtual machine rather than running directly on a host OS [S3]. • Container Technologies: Docker and to a smaller extent LXC dominate, other trending ones are covered (Kubernetes, CoreOS, OpenVZ, Diego, Rocket). LXC can be considered the de-facto standard for containerisation on Linux/Unix, while Docker has gained momentum and it is becoming the reference technology for containerisation. Open-sourcing, a rich ecosystem with image repositories and community support are here the main drivers of success. The popularity of Docker is probably linked to the open source approach and early release of technology. Though, as Bernstein points out [S5], a more neutral governance/collaboration structure around Docker (a start-up company) and Kubernetes (still controlled by Google) will allow an agreement on a wider common packaging and deployment approach. An industry perspective is added in [S8] by VMware, demonstrating an OVF-based standardised containerisation approach. Standard containers that cleanly separate application providers from infrastructure providers are a key success factor (Kecskemeti et al., 2016). • Container Type : Here, we see largely containers, then clusters (confirming above observations). However, looking at publication times, we could image that containers are more popular than clusters because the latter has been only started very recently with publications almost exclusively from mid 2015, e.g., [S9,S15,S43,S44].

Fig. 14: Study Distribution by Cloud Delivery Model

Fig. 15: Study Distribution by Container Technology

Fig. 16: Study Distribution by Container Type

Summary: We can note that container-based solutions target both IaaS and PaaS, with a consolidated interest for singlecontainer solutions, and a growing interest for solutions for clusters of containers. Managing qualities such as performance and resource utilisation are key concerns.

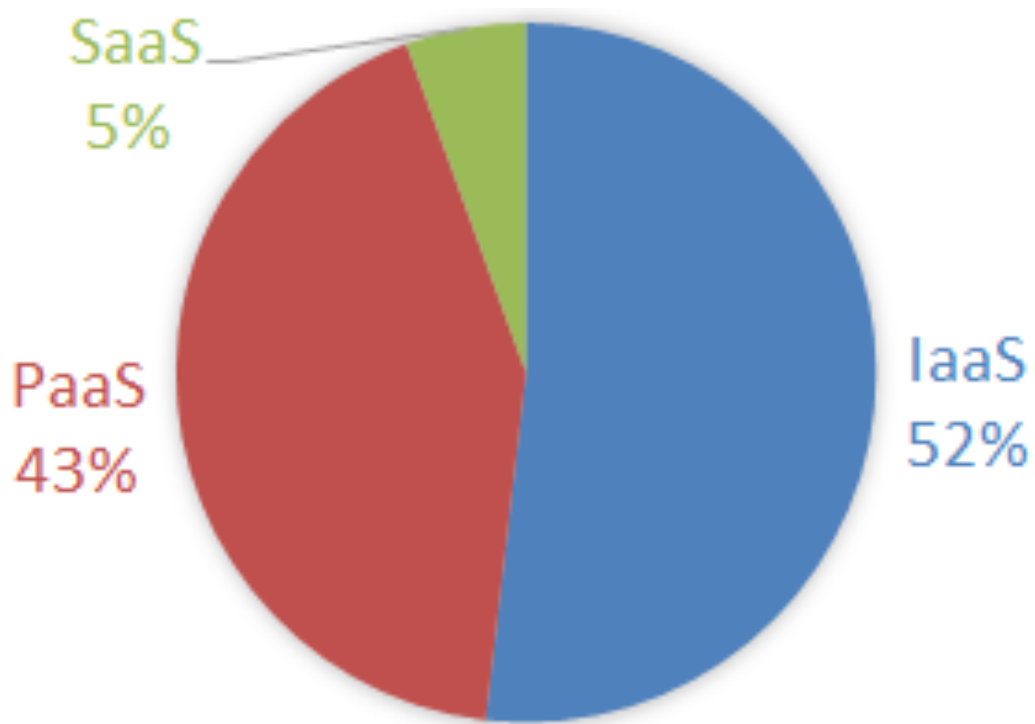


图 14：研究按云交付模型分布。

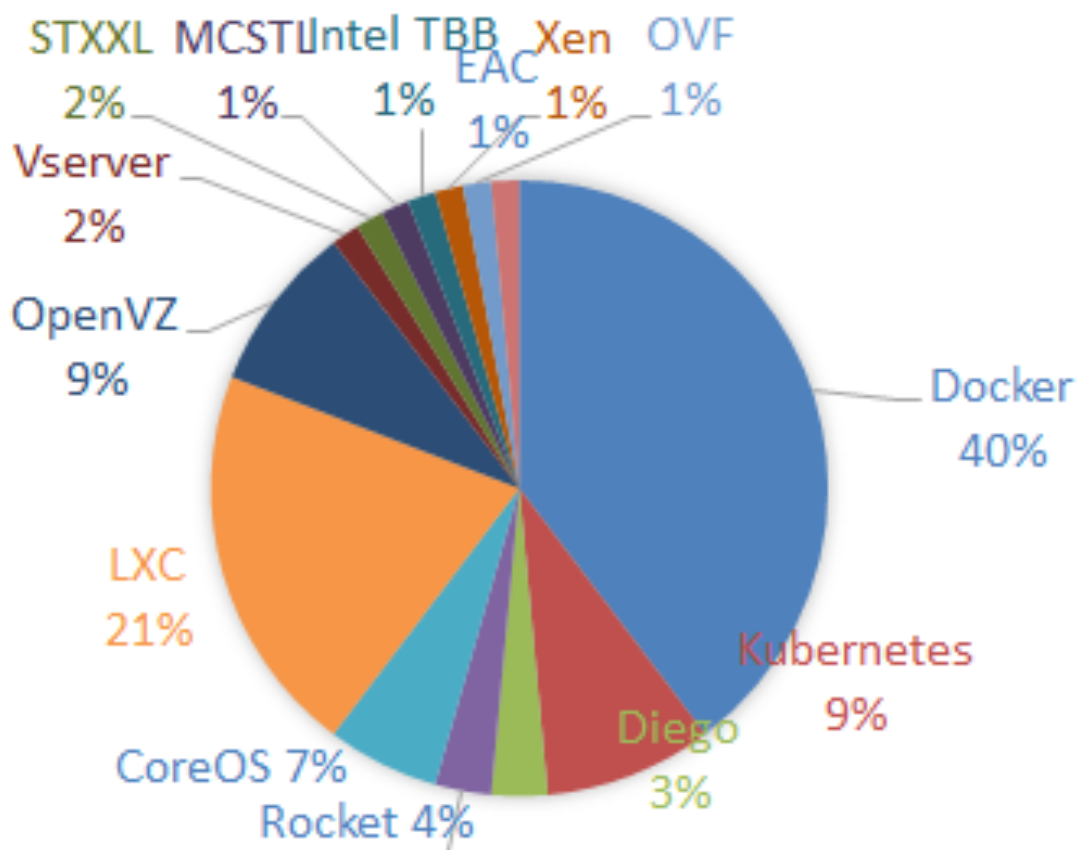


图 15：研究按容器技术分布。

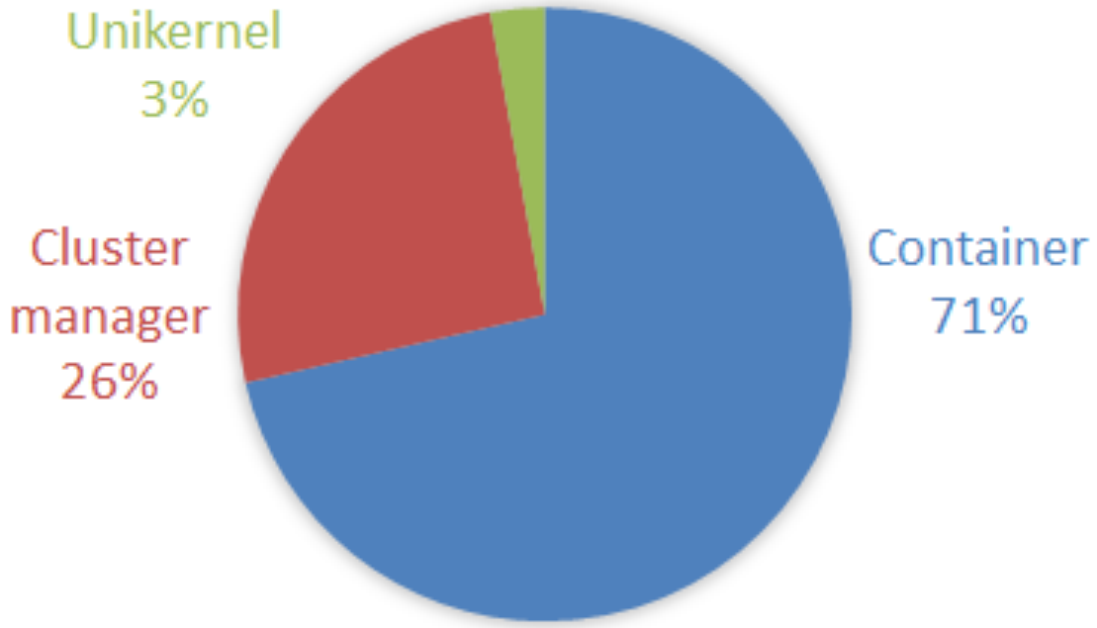


图 16: 研究按容器类型分布。

Fig. 17: Contribution Type by Stack Concern. [Sol:Solution,

Val:Validation, Eval:Evaluation, Exp: Experience Report, Rev: Review], [VB: Virtualisation Basics, ConC: Container Construction, ConM: Container Management, CluC: Cluster Construction, CluM: Cluster Management]

Fig. 18: Contribution Type by Management Service.

[Sol:Solution, Val:Validation, Eval:Evaluation, Exp: Experience Report, Rev: Review] , [Const: Construction/Architecture, Exec: Execution, QuMon: Quality-Monitorable, QuNMon: Quality-Non-Monitorable, QuTest: QualityTestable]

C. Completing the Conceptual Map

Furthermore, we looked at the Technology Stack and Management Services categories in more detail, as these were only summarised in the initial study summary in Table IV, where we compared the studies based on some core classification categories. In Figs. 17 and 18, we mapped the generic contribution types (Solution, Evaluation, Experience Report, Review) to the two above specific technical categories relevant in the container orchestration context. The bubble size indicates the number of studies of that type. The Technology Stack view, Fig. 17, shows how containers and container-based clusters are internally constructed: • mainly solutions are reported (largest bubbles), which primarily cover container construction and management, but also address all other aspects to some extent; • evaluations focus on construction, less on management; • equally, experimentation is construction-focussed; • reviews cover the technology stack more completely. In the Management Services view, Fig. 18, similar observations can be made: • mainly solutions are reported, which here more clearly focus on container execution and quality management

(regarding the latter, monitorable aspects prevail); • evaluation, experimentation and examples are patchy, focussing on the quality management; • reviews cover all service types more comprehensively. Summary: The conceptual map links the studies with the classification scheme. For each term, we recorded the number of occurrences of the extracted terms from the studies. Higher numbers of occurrences in each of the categories indicate a stronger research concern and/or stronger consensus on the importance of the aspect. In Fig. 19, the term occurrences are associated to the technical concerns of the classification scheme. The figure summarises earlier discussions and defines the cloud container technologies space into • internal construction of containers and their clusters, • methodological support for construction and architecture, • infrastructure services for development, execution management and quality control, • technology and tool platforms. The relative weight of the individual concerns within each of the four core categories is given. This shows the current focus on developing solutions for runtime management of execution and quality, but also the need to: • (methodological aspect) evaluate and experiment more, • (technical aspect) integrate the runtime focus more with development aspects into a coherent framework, • (quality aspect) broaden the quality concerns beyond the monitorable ones. Particularly the Architecture and Management categories define what are the expected functions of a cloud container technologies (with methods and techniques).

Fig. 19: Key terms extracted, organised into different concerns

and presented with the numbers of times these terms occur.

D. Threats to Validity

We discuss threats to the validity of this work in the different mapping study steps. Threats to the relevance of the selected studies. Innovation in the area emerges from industrial practice. While industrial white papers or blogs as mechanisms to communicate the

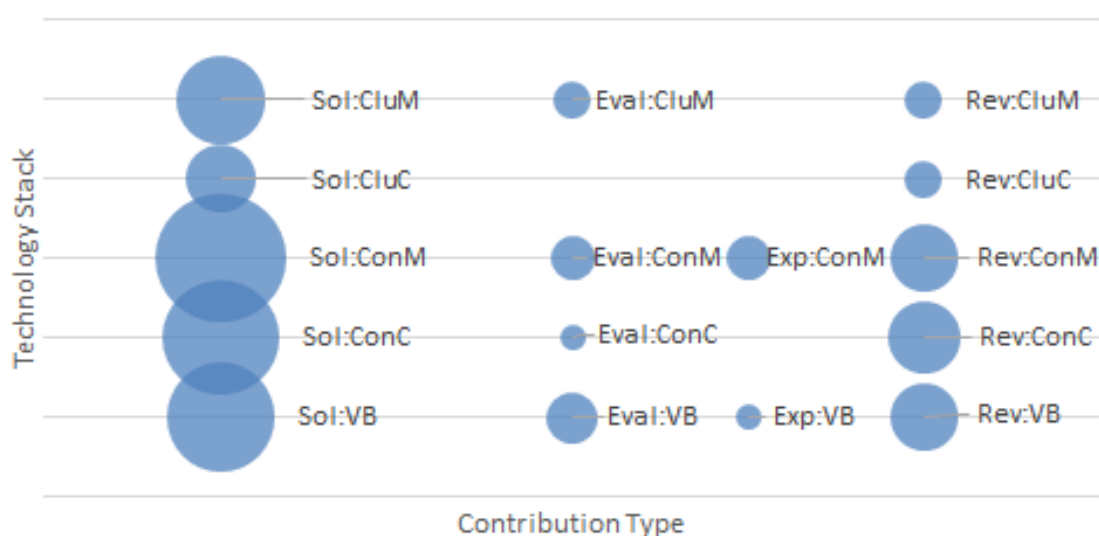


图 17: 按技术栈关注点划分的贡献类型。

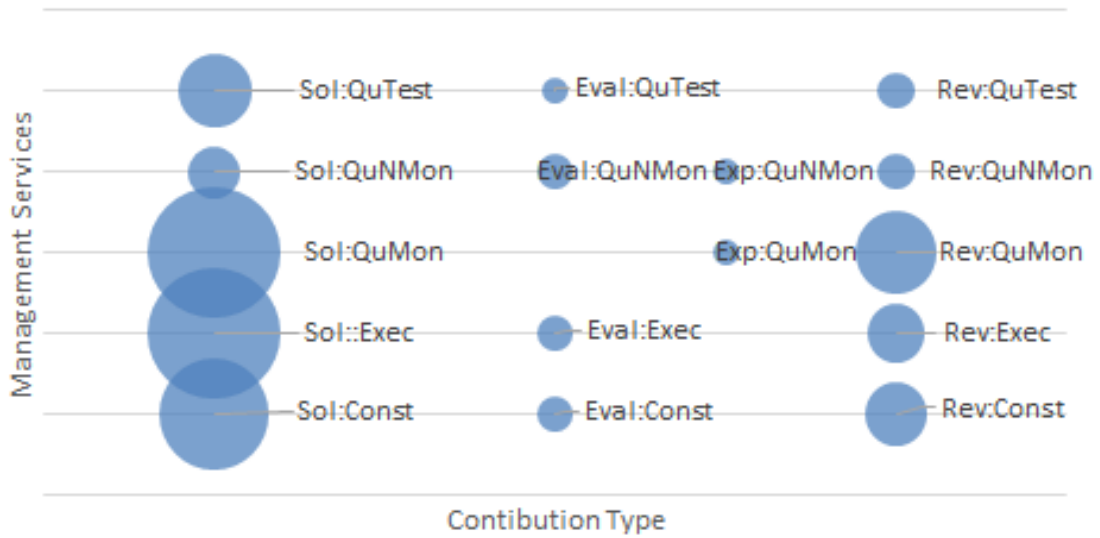


图 18: 按管理服务划分的贡献类型。

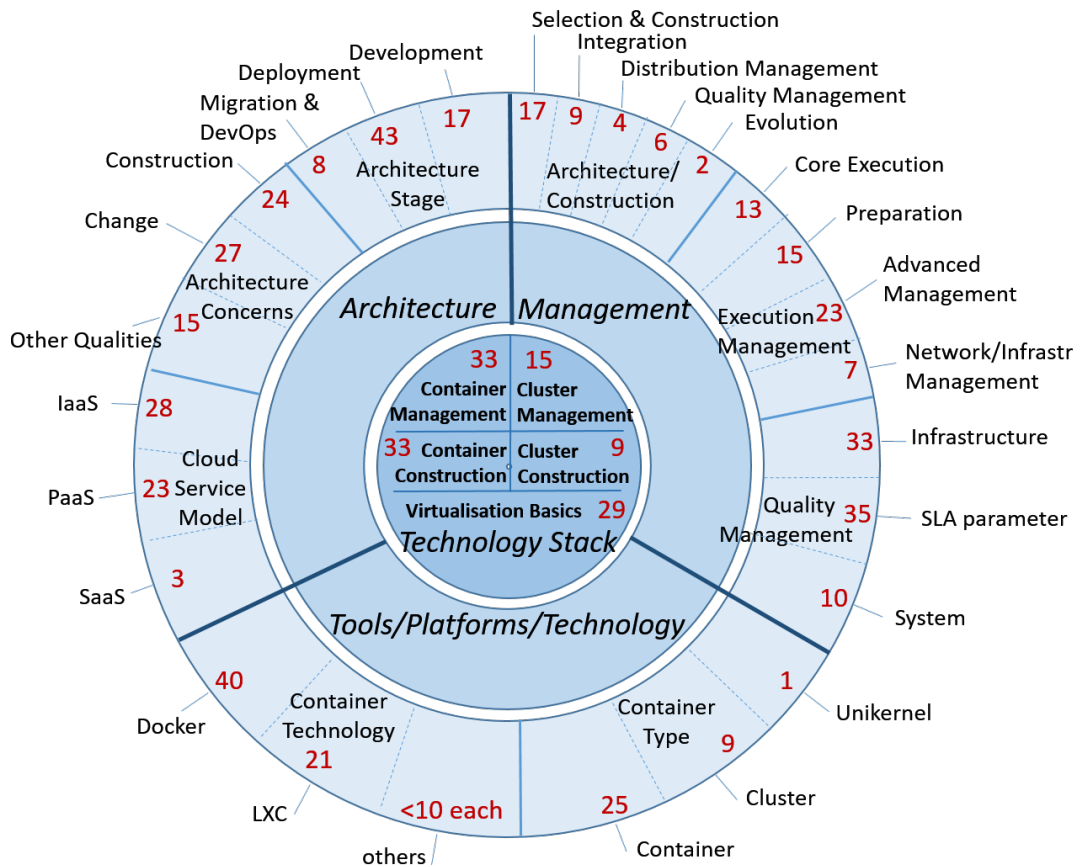


图 19: 按不同关注点组织的关键术语及其出现次数。

innovations are available, these are difficult to systematically retrieve and quality assess through SLRs or SMSs. We have therefore complemented the protocol-based results by reviewing relevant technologies such as Docker, cloud and cloudnative architectures and including exemplary blogs in the discussion. Threats to the identification of primary studies. In our search, we aimed to retrieve as many studies as possible to avoid any bias. A challenge was to determine

the scope of our study, since container orchestration relates to different computing and IT communities including software engineering, distributed systems, operating systems, information systems and cloud. These communities use different terminologies for the same concepts. To cover all and avoid any bias, we searched for the container orchestration term in different contexts. While this approach decreases bias, it significantly increases search effort. To identify relevant studies and ensure an unbiased selection, a review protocol was developed. Threats to selection and data extraction consistency. The formulation of the research questions has helped in selecting studies of relevance, as did the Characterisation Framework. However, we did include peer-reviewed magazine contributions and books here to capture trends and activities. Threats to data synthesis and results. This reliability threat is mitigated as far as possible by having a unified characterisation scheme and following a standard protocol where several steps were piloted and externally evaluated.

VI. CONCLUSIONS

Containerisation provides cloud application management based on lightweight virtualisation. The increasing interest in cloud container technologies shows the importance of their management and orchestration in this context. The scientific contributions we reviewed are a mix of technology reviews, solutions and use case architectures (conceptual and implemented). As a good part of this is still conceptual, it can be seen as a sign of the immaturity of an emerging field. This interpretation is strengthened by the fact that only some use case validations and no large-scale empirical evaluations exist. There is also a noticeable imbalance of contribution formats compared to more mature domains:

- Larger number of technical contributions (solution proposals): the number of journal publications is low (with short communications in magazines published only).
- Higher number of use cases than technology solutions: i.e., an emerging technology to be formatively validated through use cases rather than summative evaluations. What has been demonstrated is the better resource efficiency of containers compared to VMs, and also the increased flexibility as an application management framework. However, proven technologies are lacking to fully support container architectures for cloud environments. An example pointed out in [S21] is failure management. Finding root causes and dealing with anomalous events requires better resource monitoring and log analysis. We can conclude that the field is moving towards container middleware (and even container PaaS) with isolation, construction, quality management, orchestration and distribution management as core concerns of a container PaaS middleware, but key features such as failure management are still missing. We see here middleware as the core set of features necessary to host and provide applications, facilitating the application and Web tier. A PaaS is a more comprehensive solution encompassing (virtualised) infrastructure features and data(base) storage. Container orchestration is a key concern in the current cloud PaaS context. For instance, the Cloud Native Computing Foundation CNCF (<https://cncf.io/>) will be integrating the

orchestration layer of the container ecosystem. The CNCF uses Kubernetes container cluster orchestration as its containerisation technology to deal with network storage and cluster management. Industry reports, such as [S21] about Google's Borg cluster management, highlight current limitations - e.g., Borg has no first-class mechanism to manage an entire multi-job service as a single entity. Better cluster orchestration support is the required solution. Management becomes inherently distributed, with features like the scheduler, admission control, vertical and horizontal auto-scaling, re-packing, periodic job submission, workflow management and archiving. The aim is to scale up the workload and feature set without sacrificing performance or maintainability. Organizations are packaging applications in containers and need to orchestrate multiple containers across cloud servers:

- The benefits of container-based orchestration include adjustable cluster sizes for the deployment of containers, easier cluster maintenance and also quicker deployment. This is confirmed by the motivations cited, which prioritise the easy, automated container deployment and management in larger settings allowing for flexible migration and reconfiguration.
- Containerisation positively impacts on both development and deployment aspects such as testing and monitoring of container-based applications, which we found both covered by the studies in terms of a mix of architecture design and continuous, quality-driven management.
- The quality distribution shows a strong interest in optimised resource utilisation (effectively a cost factor) and performance, complemented by portability/interoperability and security. Interesting is also the emergence of two equally important perspectives: quality aspects directly observable by the consumers and the quality aspects relevant to the management of the applications within the platform itself. A theme that emerges is the support of continuous development through containers, joining both construction as well as operations and management.

In the cloud, cloud-native platform services for development and deployment do exist, but require advanced PaaS orchestration support. The trend towards cluster-based orchestration, combined with the interoperability of successful container technologies, also allows the management of highly distributed topologies of smaller virtualised devices beyond centralised clouds as in the edge/fog cloud domain [S9,S21,S25,S26,S43,S44]. Containers can run on single-board devices and can be deployed on clusters of these devices, making them suitable for edge and IoT computing (Pahl et al., 2016).

This results in a need for research beyond the performance and isolation concerns for container virtualisation, particularly regarding methodological and tool support :

- A DevOps approach to manage the continuous development and deployment would benefit - the cross-stage nature is obvious. While most papers look at the two sides separately, the need to link these in a continuous process and to feed back monitored operational data into development becomes obvious.
- Containers represent a progression from virtual machines towards lightweight application management. Lately, there is an observable continuing trend towards serverless

architectures and other mechanisms to manage orchestration and deployment complexity such as unikernel technology towards more lightweightness. • Similar to Docker kick-starting research on container technology as an active open-source environment, a similar impact may be expected with the emergence of similar technologies in the cluster space such as Kubernetes and Mesos. There is a strong increase in research papers from mid 2015 that evidences this. Edge or fog computing is an architectural setting that needs clustering in which containers can help to address interoperability needs.

ACKNOWLEDGEMENTS

The work presented here was partly supported by project PRA 2016 64 "Through the fog" funded by the University of Pisa and by project "Edge Cloud Orchestration" funded by the Free University of Bozen-Bolzano.

REFERENCES

- Brogi, A., Carrasco, J., Cubo, J., D'Andria, F., Di Nitto, E., Guerriero, M., Pérez, D., Pimentel, E., and Soldani, J. (2016). SeaClouds: An Open Reference Architecture for Multi-cloud Governance, volume 9839 of LNCS, pages 334–338.
- Dupont, C., Sheikhalishahi, M., Facca, F. M., and Cretti, S. (2016). Energy efficient data centres within smart cities: IaaS and PaaS optimizations. In *Smart City 360*, pages 408–415.
- Felter, W., Ferreira, A., Rajamony, R., and Rubio, J. (2015). An updated performance comparison of virtual machines and Linux containers. In *IEEE Intl Symposium On Performance Analysis of Systems and Software (ISPASS)*, pages 171–172.
- Jamshidi, P., Ahmad, A., and Pahl, C. (2013a). Cloud migration research: a systematic review. *IEEE Transactions on Cloud Computing*, 1(2):142–157.
- Jamshidi, P., Ghafari, M., Ahmad, A., and Pahl, C. (2013b). A framework for classifying and comparing architecture-centric software evolution research. In *7th European Conference on Software Maintenance and Reengineering*, pages 305–314.
- Kecskemeti, G., Marosi, A. C., and Kertesz, A. (2016). The ENTICE approach to decompose monolithic services into microservices. In *Intl Conf on High Performance Computing Simulation (HPCS)*, pages 591–596.
- Kitchenham, B., Brereton, O. P., Budgen, D., Turner, M., Bailey, J., and Linkman, S. (2009). Systematic literature reviews in software engineering – a systematic literature review. *Information and Software Technology*, 51(1):7–15.
- Kratzke, N. (2015). About microservices, containers and their underestimated impact on network performance. *Conf on Cloud Computing*, 2015.
- Lewis, J. and Fowler, M. (2014). Microservices. <http://martinfowler.com/articles/microservices.html>.

- Liu, C., Loo, B. T., and Mao, Y. (2011). Declarative automated cloud resource orchestration. In Proceedings of the 2nd ACM Symposium on Cloud Computing, page 26. ACM.
- Manu, A., Patel, J., Akhtar, S., Agrawal, V., and Murthy, K. (2016). Docker container security via heuristics-based multilateral security-conceptual and pragmatic study. In Intl Conf on Circuit, Power and Computing Techn, pages 1–14.
- Mell, P. and Grance, T. (2011). The NIST definition of cloud computing.
- Pahl, C. (2015). Containerization and the PaaS Cloud. IEEE Cloud Computing, 2(3):24–31.
- Pahl, C., Helmer, S., Miori, L., Sanin, J., and Lee, B. (2016). A container-based edge cloud PaaS architecture based on Raspberry Pi clusters. In 4th IEEE Intl Conf on Future Internet of Things and Cloud Workshops.
- Petersen, K., Feldt, R., Mujtaba, S., and Mattsson, M. (2008). Systematic mapping studies in software engineering. In Intl Conf on Evaluation and Assessment in Software Engineering.

二、中文译文

云容器技术：研究现状综述

文献来源：Pahl C, Brogi A, Soldani J, Jamshidi P. Cloud Container Technologies: A State-of-the-Art Review[J]. IEEE Transactions on Cloud Computing, 2019, 7(3): 677-692. DOI: 10.1109/TCC.2017.2702586.

摘要

容器作为一种轻量级的应用虚拟化技术，近年来取得了显著成功，尤其适用于云环境中的应用管理。很多情况下，容器集群的管理变得十分必要，而容器构建与部署过程的编排也随之成为核心问题。尽管这一新兴主题已经受到研究者关注，但目前仍缺乏能够对相关研究进行系统整合的二次研究。本文旨在识别、分类并系统比较现有关于容器及其编排的研究成果，尤其关注该技术在云环境中的应用。为此，作者对 46 篇入选研究进行了系统映射研究，并基于特征化框架对这些文献进行了分类和比较。研究结果围绕容器编排领域中已经形成共识的问题以及正在出现的新问题展开讨论，将其放在云计算背景下加以定位，同时也使其更接近当前云平台、微服务以及持续开发等热点议题。

关键词

编排，容器，集群，云，系统文献综述，系统映射研究。

一、引言

容器化是一种以轻量级方式实现应用虚拟化的技术，这种技术已经在云应用管理中得到广泛采用。如何对单个容器以及容器集群的构建与部署进行编排，已经成为一个核心问题。当前尚缺少针对云环境中容器技术研究的二次研究，这使得研究者难以从整体上评估该领域的发展成熟度，也不利于识别其中的发展趋势、研究空白以及未来方向。随着容器

及其在云中管理和编排问题受到越来越多的关注，有必要对现有研究进行系统梳理。

二次研究的作用在于识别、分类和综合已有研究成果，从而形成对研究现状的比较性总体认识，并对正在开展的工作进行评价。作者选择采用系统映射研究方法，是因为这一方法更适合用来梳理和构建一个新兴研究领域。本文对容器技术及其在云环境中的应用进行了识别、分类和系统比较，试图从中进一步理解平台即服务（PaaS）在两个层面的意义：一是基于容器实现应用封装的中间件平台，二是支撑应用部署的基础设施平台。

本文共选取了 46 篇研究文献，时间跨度超过十年，从 2007 年开始。作者基于一个特征化框架对这些文献进行了分类与比较，并由此形成了一个关于当前研究方法、技术手段、最佳实践以及经验总结的知识库，重点关注云架构中的应用开发与应用管理问题。研究结果表明，容器技术研究仍然处于形成阶段，还需要更多实验性和经验性的评估来验证其实际收益。同时，当前在工具支持方面仍存在不足，尤其缺乏能够自动化、便利化地支撑容器管理与编排的工具，特别是在云集群架构下更为明显。

系统映射研究还表明，以 LXC、Docker 为代表的容器技术正在被越来越广泛地使用。一方面，它们作为基础设施即服务（IaaS）层面的轻量级虚拟化方案；另一方面，它们又作为平台即服务（PaaS）层面的应用管理方案。可以看出，容器技术对应用开发和部署两个方面都产生了积极影响。例如，云架构设计正在向基于 DevOps 的方式演进，从而支持持续开发与持续部署流水线，并逐渐采用基于容器及其编排机制的云原生架构方案。研究结果说明，容器能够在云平台服务支持下促进持续开发与持续部署，但同时也确实需要更高级的编排机制来配合。

因此，基于容器的编排技术逐渐成为一种在云化集群环境中组织计算资源的重要机制。研究结果还表明，这类技术被视为平衡技术质量管理需求的一种有效方式，例如能够更好地优化资源利用率与系统性能，而这些因素在按需计费的云环境中直接关系到成本控制。本文的系统映射研究首先有助于软件工程、分布式系统以及云计算领域的研究者识别相关研究；其次，系统化呈现研究成果有助于理论构建、方案设计、研究影响分析以及未来研究方向的建立；此外，它也能帮助工程实践人员理解当前可用的方法、技术和工具，以及这些方案的约束条件和成熟程度。文章其余部分中，第二部分介绍背景与相关研究，第三部分说明研究方法、研究问题与研究范围，第四部分提出云容器编排的特征化框架，第五部分给出系统映射结果及其局限性分析，第六部分讨论研究发现、启示与发展趋势。

二、容器架构及其管理

云计算通过虚拟化技术实现大规模共享资源的弹性调度。传统上，虚拟机通常是基础设施层的核心支撑。而容器化则不同，它通过从镜像仓库中获取单个镜像，并据此定制构建容器应用包，实现了一种更轻量级的虚拟化方式。与虚拟机相比，容器在资源消耗和启动时间方面都更具优势，同时也能提供云中可移植、可互操作的软件应用所需的更高水平的应用封装能力。容器化依托于一种能力：开发者可以将应用开发、测试并部署到大量服务器上，同时还可以把这些容器互联起来。因此，容器技术主要对应云平台即服务层面的问题。鉴于云计算本身的重要性，对当前相关活动进行一种整合性的观察显得十分必要。

2.1 容器技术原理

容器中封装的是应用的自包含、可直接部署的组成部分，如果有需要，还包括运行应用所需的中间件与业务逻辑，以及相应的二进制文件和库。像 Docker 这样的工具建立在容器引擎之上，其中容器充当一种可移植的应用打包手段。这样一来，在多层应用场景下，容器之间的依赖关系就必须得到有效管理。一个编排方案可以描述系统组件、组件依赖以及它们在分层结构中的生命周期。随后，PaaS 云平台可以通过代理机制，例如容器引擎，去执行这个方案中的工作流。因此，PaaS 平台能够基于容器完成应用部署。这里所谓的编排，是指对容器的构建、部署以及持续运行过程进行协调一致的管理。

很多容器方案都基于 Linux 的 LXC 技术。近年来的 Linux 发行版，作为 Linux 容器项目 LXC 的组成部分，提供了命名空间和 cgroups 等内核机制，用于在共享操作系统之上对进程进行隔离。当前最流行的容器方案是 Docker，因此作者用它来说明容器化的实现方式。Docker 镜像由多层文件系统叠加而成，类似于 Linux 虚拟化栈。Docker 使用联合挂载机制，在只读文件系统的顶部增加一个可写文件系统层，从而允许多个只读文件系统相互堆叠。利用这一特性，开发者可以在基础镜像之上继续构建新镜像，而最上层的可写部分就是容器本身。

容器化进一步推动了系统从“单个应用运行于容器中”向“多个容器主机构成集群并跨主机运行容器化应用”的演进。后者充分利用了容器天然具备的互操作性。多个单独的容器主机会被组织成互联集群，每个集群由若干主机节点构成。应用服务通常表现为来自同一镜像的一组逻辑容器集合，这种组织方式使得应用能够在不同主机节点之间进行扩展。对于需要持久化数据的应用，还可以使用卷机制，容器通过挂载这些卷来进行数据存储。链接机制则允许两个或多个容器之间建立连接并进行通信。要完成这类容器集群的部署和管理，就必须依赖编排机制来支撑容器间通信、容器链接以及服务组合。

2.2 基于云的容器架构

容器编排不仅仅是简单地打开或关闭应用，也不仅仅是启动、停止容器或在服务器之间迁移容器。作者将编排定义为：对可能呈分布式部署的、基于容器的软件应用集群进行构建和持续管理。容器编排允许用户在部署一个多容器应用时，定义这些容器在云环境中的协同方式。它不仅负责容器的初始部署，还负责把多个容器作为一个整体来进行后续管理，包括可用性、伸缩性以及网络连接等方面。实质上，云环境中的容器构建本身就是一种分布式云体系结构中的编排行为。

云可以被看作一种分布式、分层的架构体系，其中核心基础设施层、平台层和软件应用层分布在多云环境之中。容器技术在这里发挥着重要作用。可以预见，容器技术将在未来应用管理中占据核心地位，尤其是在云平台即服务场景下更是如此。近年来流行的微服务架构也可以通过容器在这一云框架中得到实现。随着架构设计理念发生这样的变化，对该领域开展系统性二次研究能够帮助工程实践人员更好地做出技术选择判断。

2.3 研究现状

本文用于评估云容器技术的方法是系统映射研究。相关综述研究大体可以分为两类。第一类是系统文献综述（SLR），它更适合对已经较为成熟的研究领域进行总结性分析，通常建立在更大规模文献基础之上。第二类是系统映射研究（SMS），它更适合分析研究报告的结构类型和分布情况，并进行可视化分类，尤其适用于高质量原始研究还不够丰富的领域。与 SLR 相比，SMS 的细节通常更少，但更适合本文的研究目的。

在文献筛选过程中，作者提取出了 6 篇与研究目标相关的综述性论文。其中有 5 篇可以归类为技术综述，即主要对容器技术本身进行概述和评估。文献 [S3] 讨论了虚拟化基础以及容器的构建与管理，更偏重于部署层面；文献 [S4] 主要讨论虚拟化基础，但同时兼顾部署和开发视角；文献 [S6] 的内容更为全面，涉及集群问题，并同时涵盖部署与开发两个角度；文献 [S9] 与 [S6] 类似，但对质量管理问题涉及较少；文献 [S19] 与 [S4] 一样关注虚拟化基础，但更加聚焦于高性能计算以及计算密集型、存储密集型应用中的性能问题。稍有不同的是文献 [S17]，它更多是围绕研究覆盖面来组织内容，而不只是围绕技术本身展开。然而，像本文这样对相关文献进行系统性覆盖和比较的研究仍然缺失。系统文献综述和系统映射研究的核心，在于通过一个特征化框架，对云环境中容器技术应用的现有证据进行识别、分类和比较。正如前文所指出的，虽然已有一些技术综述存在，但这些工作主要聚焦技术介绍，并没有系统地揭示该领域的研究努力与发展方向。

三、研究方法

3.1 系统映射过程

系统映射研究通过一套严格的方法步骤来检索和分类文献，从而尽可能降低研究偏差。它依赖经过明确设计和评估的评审协议来完成数据提取、分析和结果记录。本文遵循 Petersen 等人在 2008 年提出的三步审查流程，包括规划、实施和记录三个阶段。同时，作者还对每一步的结果进行了评估，并为研究背景额外构建了一个特征化框架。

作者将系统映射方法应用到了以容器编排为重点的云技术研究中。该系统映射研究的关键步骤包括：定义研究问题、检索相关文献、筛选文献、对摘要进行关键词提取，以及数据提取与映射。每一步都会产生相应成果，而整个流程的最终产出就是系统映射图。根据研究目标，作者首先明确研究问题 and 研究范围，以便据此构建文献检索字符串。

此外，作者还利用 PICO，即 Population、Intervention、Comparison、Outcome 这四个维度来界定研究的总体目标与范围。评审过程从文献选择开始，最终得到提取后的数据和综合后的信息。本文特别聚焦“编排”这一概念，是因为作者希望从研究视角把握容器架构向分布式方向发展的趋势，因此将 *orchestration* 作为一个宽泛、包容性较强的术语来使用。

3.2 研究问题的定义

在下一步工作中，作者通过定义研究问题来塑造评审协议。系统映射研究的主要目标，是对某一研究领域形成总体概览，并识别其中已有研究的数量、类型以及研究成果。研究者还可以通过统计不同年份的发表频率来发现发展趋势。另一个次级目标是识别相关研究主要发表在哪些论坛、期刊和会议上。这些目标最终都体现在本文提出的研究问题之中。

3.3 原始文献检索

检索词的选择依据 Petersen 等人的方法，并以研究问题为指导。原始研究通常通过在科学数据库中使用检索字符串来识别，也可以通过人工浏览会议论文集或期刊来补充。构造检索字符串的一种常见方法，是基于 PICO 结构来组织关键词，从而使检索词与研究问题保持一致。检索字符串中的关键词可以分别来自 PICO 结构的不同方面。需要指出的是，与 Petersen 等人的建议不同，本文并未将特定结果类型或实验设计类型纳入限制条件。这样做是因为作者希望获得对整个研究领域更宽泛的总体认识。如果一开始就只限定某些特定类型的研究，可能会导致研究概览带有偏差，同时也会造成系统映射结果不完整。某些子主题还可能因为研究方法不同而出现过度代表或代表不足的现象。

表 1 研究问题

RQ1 关注“研究应用”：即基于容器的方法被应用在了哪些云活动中、出于什么原因被采用、以及具体是如何应用的。作者指出，容器既可以被看作基础设施层中虚拟机的一种替代方案，也可以被看作与平台服务和软件服务相关的应用封装机制。因此，需要围绕核心架构问题建立一个系统映射，包括五个方面：一是动机，即在云中采用容器的预期收益是什么；二是技术栈，即容器化系统在教育层和平台层是如何构建的；三是管理服务、云环境与体系结构，即这些系统是如何被开发和管理的；四是技术空间，即构建和管理过程中具体使用了哪些云和容器技术；五是应用领域，即云中的容器究竟被用于哪些实际场景。

RQ2 关注“研究分布”：即关于云活动中容器技术的研究主要发表在哪些来源中、在什么时间发表。由于这一主题同时涉及云计算、软件工程、分布式系统和操作系统等多个学术共同体，因此相关研究理论上应当分散在多个发表渠道中。了解这些信息，有助于识别核心发表平台，也有助于观察该主题相关研究数量的变化趋势。

RQ3 关注“成熟度”：即该领域目前的发展成熟程度如何。研究方法和评估方式能够反映该领域的成熟状态，例如，是否已经出现了足够多的重要实证研究来证明其实际价值，还是目前的研究重点依然停留在技术问题探索阶段。与此同时，不同贡献类型之间的比例关系，例如解决方案提案、经验报告和综述论文之间的关系，也有助于判断该领域的成熟度。

RQ4 关注“发展趋势”：即当前研究主要关注哪些问题，未来研究议程应当如何展开。其目的在于揭示研究空白，并识别未来的发展方向。作者认为，这仍然是一个较新的研究主题，整个领域还处于不断成熟之中，因此自然会产生两个问题：当前还有哪些开放问题，以及未来还面临哪些挑战。文中还指出，这一点也体现在检索字符串之中，即 ``(cloud* OR PaaS) AND (container*) AND (orchestrate* OR cluster* OR manage*)``，其中星号代表词汇上的相关变体。基于 PICO 准则，作者最终选择从“技术视角”出发构造上述检索式。最初还考虑过其他 PICO 类别，但未纳入最终检索式之中，例如产品视角中的 Docker、Kubernetes、Diego、Rocket、LXC 等关键词，以及“实验性”“经验性”“技术性”等干预维度，还有 framework、theory、architecture、design、language、use case、case study 等结果维度。作者没有把这些内容直接纳入检索条件，是为了避免检索范围过窄而造成遗漏，但在后续的纳

入与排除标准制定中仍然考虑了这些因素。

由于这一研究议题在云计算领域出现较晚，相应的发表论坛可能尚未被数据库完整收录，因此作者在正式筛选之前采用了一个初始范围更宽、并且部分依赖人工检索的策略。作者先通过较宽泛的检索式建立整体文献集合，再逐步将其收缩到更聚焦、更高质量的范围内。最终纳入考虑的数据库包括 IEEE Xplore、ACM Digital Library、Science Direct、ISI Web of Science、SpringerLink、INSPEC、EI Compendex 和 DBLP。考虑到该领域较新且存在索引不完整的问题，Google Scholar 在正式纳入与排除阶段之前的初始筛选过程中发挥了关键作用。

3.4 纳入与排除标准

在完成候选文献的初步收集之后，作者通过设定纳入与排除标准来对结果集进行进一步筛选。纳入的研究需要满足以下条件：明确讨论云环境下基于容器的虚拟化问题；在平台层面涉及编排、集群或管理；并且能够给出具体的技术方案、实证评估或结构化的工程经验。被排除的研究主要包括：缺少足够技术深度的短篇工作坊笔记；缺乏证据支撑的观点性文章；以及那些只把容器作为边缘性实现细节、而非作为主要研究对象的论文。来自不同数据库的重复条目会被合并；同一项研究的会议初版与后续期刊扩展版会被整合，并仅保留时间更近、内容更完整的版本进行后续分析。通过这一过程，候选研究集合最终被收敛为 46 篇核心研究，作为后续分类与比较工作的基础。

3.5 分类与数据提取

随后，作者将 46 篇入选研究与第四节提出的特征化框架进行映射。对于每一篇研究，分别记录其发表来源、发表年份、贡献类型、技术栈、评估方法以及主要关注的问题。贡献类型被划分为解决方案提案、经验报告、技术综述以及概念性研究四类；评估方法被划分为实验室实验、案例研究、工业应用以及分析性讨论四类。分类过程由至少两位作者独立完成，对存在分歧的条目则通过讨论达成一致。最终形成的数据集既可以支持定量分析，例如观察各类贡献随时间的分布频率；也可以支持定性分析，例如归纳围绕编排、安全和微服务等问题反复出现的主题。这一系统化过程使得映射结果具有可重复性，并为第六节关于趋势和研究空白的讨论提供了稳定的基础。

图 16：研究按容器类型分布。

图 17：按技术栈关注点划分的贡献类型。

图 18：按管理服务划分的贡献类型。

图 19：按不同关注点组织的关键术语及其出现次数。

3.6 摘要关键词提取、数据提取与系统映射

在分类方案形成之后，实际的数据提取工作就是把相关研究按照该方案进行归类。分类框架并不是一次性固定不变的，而是在数据提取过程中根据新增材料和外部专家反馈持续演化，具体包括新增类别、合并相近类别以及拆分过于宽泛的类别。研究团队将数据记

录在电子表格中，以便跟踪每篇文献在各分类维度上的归属，并据此统计不同类别中研究发表的频率。通过这种方式，作者能够识别过去研究中被反复强调的主题，也能够发现仍然薄弱、值得进一步研究的方向。

四、云容器技术分类框架

作者首先提出了一个以架构为中心的参考模型，用来对云容器技术进行分类。该模型的目的，是在概念层面展示当前研究的覆盖情况，并进一步识别研究趋势和未来方向。分类框架将常见的软件工程术语、方法支持、架构关注点、工具支持和应用场景组织在一起，用于系统地归类前文筛选出的 46 篇原始研究。

该框架的顶层类别与研究问题保持对应关系，主要包括技术栈、架构、管理服务以及工具、平台和技术等方面。其中，技术栈关注容器技术本身及其内部机制；架构、管理服务与工具平台则关注容器被使用和被支持的云环境。具体子类是在第一轮阅读入选研究后提炼出来的，目的是保证框架能够覆盖真实文献中的研究重点。最后一列中的具体术语来自每篇研究的摘要和正文，作者再根据语义关系将这些术语人工组织到相应的类别中。

在初始形成之后，分类框架经历了作者之间的多轮迭代，并由来自三个国家、三个组织的五名外部专家进行验证。专家反馈促使作者对框架的部分分类进行修正和补充，从而提高了分类方案的可靠性和可解释性。

五、结果与可视化

第五部分给出了系统映射研究的主要结果。作者首先利用表格概览所有入选研究在关键类别上的分布，再分别从通用研究属性和领域特定技术属性两个角度展开分析。通用属性包括发表年份、发表渠道、研究共同体、发表形式、贡献类型和评估方法；领域特定属性则包括采用动机、技术栈、管理服务、架构设置、云交付模型、具体容器技术和容器类型。

5.1 原始研究的通用属性

从时间分布看，早期关于容器编排的研究数量较少，Docker 等工具出现后研究数量显著增加，说明容器技术开始成为云计算领域的重要研究对象。2007 年关于基于容器的操作系统虚拟化论文被认为具有开创性，它较早将容器作为云计算方案纳入研究视野。随后，随着开源容器生态逐渐成熟，研究者才更系统地围绕容器编排、集群和管理展开研究。

从发表渠道看，相关研究主要集中在云计算领域，同时也分布于网络、分布式系统和软件工程等社区。由于容器技术发展速度较快，许多成果以会议和研讨会论文形式发表，期刊和杂志中则更多出现综述性文章。贡献类型方面，解决方案提案占主导地位，验证研究和评价研究较少，这说明该领域仍处于形成阶段。评估方法方面，已有研究更多依赖样例实现、案例分析和有限实验，缺少大规模经验报告和严格理论基础。

5.2 原始研究的领域特定属性

在采用动机方面，易部署、轻量级资源管理、大规模部署和自动化部署是最主要的驱

动因素。其中，对异构应用进行大规模、便捷部署和管理的需求，甚至比单纯的轻量级资源管理更突出。技术栈方面，研究重点集中在容器构建和容器管理，尤其是运行时管理、应用执行、通信和集群管理等问题。与此同时，容器作为一种虚拟化形式，也带来了隔离、安全、性能和操作系统支持等基础问题。

在管理服务方面，作者将相关工作分为架构与构建、执行管理以及质量管理三类。架构与构建主要关注组件组合、互操作、拓扑结构、适应性、微服务架构和迁移；执行管理主要关注编排、资源供给、启动、负载管理、配置、调度和网络地址映射；质量管理则重点涉及性能、资源利用率、启动时间、弹性、安全、可靠性、内存使用、可移植性、互操作性和可伸缩性等指标。总体来看，性能和资源利用率是最核心的质量关注点，因为它们直接影响云环境中的成本和服务质量。

在架构设置方面，研究主要集中在部署阶段，需求分析、设计阶段以及跨阶段 DevOps 主题相对较少。作者指出，部署、运行时监控、测试和持续交付之间需要更紧密的集成。云交付模型方面，IaaS 和 PaaS 几乎同等重要：容器既可以作为轻量级虚拟化机制支持基础设施层，也可以作为应用打包和管理机制支撑平台服务层。具体技术方面，Docker 和 LXC 占据主导地位，Kubernetes、CoreOS、OpenVZ、Diego 和 Rocket 等技术也开始出现。

5.3 概念映射补全

为了进一步补全概念地图，作者将通用贡献类型与技术栈、管理服务等技术类别进行交叉映射。技术栈视图表明，多数研究以解决方案为主，重点覆盖容器构建和容器管理，也在一定程度上涉及虚拟化基础、集群构建和集群管理。管理服务视图也表现出类似特征：解决方案主要集中在容器执行和质量管理，评价、实验和示例研究较为分散，而技术综述覆盖范围相对更完整。

综合来看，概念地图把入选研究与分类框架联系起来，并通过术语出现次数衡量不同关注点的重要程度。当前研究的核心集中在容器及其集群的内部构建、构建与架构的方法支持、开发和执行管理所需的基础设施服务、质量控制以及工具平台。作者由此指出，未来研究需要在三个方向加强：一是方法层面需要更多评价和实验；二是技术层面需要将运行时管理与开发过程整合成连贯框架；三是质量层面需要从可监测指标进一步扩展到安全、可移植性、互操作性和可靠性等更广泛问题。

5.4 有效性威胁

作者从系统映射研究的不同步骤讨论了有效性威胁。首先，容器领域的大量创新来自工业实践，相关白皮书和博客虽然包含重要信息，但难以像学术论文一样系统检索和质量评估，因此作者在协议化检索之外补充考察了 Docker、云原生架构等相关技术资料。其次，容器编排横跨软件工程、分布式系统、操作系统、信息系统和云计算等多个社区，不同社区对相同概念使用的术语并不一致，这会增加检索难度。为降低偏差，作者围绕容器编排在不同语境下进行检索，并设计了评审协议。

在选择和数据提取一致性方面，研究问题和分类框架帮助作者筛选相关研究，但为了

捕捉趋势，作者也纳入了经过同行评审的杂志文章和书籍内容。数据综合方面的可靠性威胁，则通过统一的特征化框架、标准化协议、多轮试点以及外部评价来尽可能缓解。

六、结论

文章认为，容器化通过轻量级虚拟化为云应用管理提供了新的基础。云容器技术受到越来越多关注，说明其管理和编排已经成为云平台中的关键问题。作者综述的研究贡献包括技术综述、解决方案和用例架构，其中相当一部分仍停留在概念或初步实现阶段，这表明该领域仍然不够成熟。只有部分研究进行了用例验证，目前还缺少大规模经验性评价。

研究结果表明，容器相较虚拟机具有更好的资源效率，也能提升应用管理框架的灵活性。但是，面向云环境的容器架构仍缺少成熟支撑技术。例如，Google Borg 集群管理经验显示，故障管理、根因定位和异常事件处理仍然需要更好的资源监控和日志分析能力。作者进一步指出，该领域正朝着容器中间件甚至容器 PaaS 方向发展，隔离、构建、质量管理、编排和分布式管理将成为容器 PaaS 中间件的核心关注点。

随着组织越来越多地将应用打包到容器中，并需要跨多个云服务器编排多个容器，容器编排的重要性持续上升。其主要收益包括：可以根据部署需求调整集群规模，降低集群维护难度，加快部署速度；能够同时影响开发和部署过程，例如测试、监控、架构设计和持续质量管理；能够围绕资源利用率、性能、可移植性、互操作性和安全性等质量指标展开优化。

文章最后指出，容器支持持续开发的趋势十分明显，它将构建过程与运行管理过程连接起来。云原生平台服务虽然已经支持开发和部署，但仍然需要更高级的 PaaS 编排能力。面向集群的编排趋势，再加上成功容器技术的互操作性，也使容器适用于边缘云、雾计算和物联网等高度分布式场景。未来研究不应只关注容器虚拟化的性能和隔离问题，还应进一步加强方法支持和工具支持。DevOps、轻量级应用管理、无服务器架构、unikernel、Kubernetes、Mesos 以及边缘/雾计算中的集群需求，都会继续推动云容器技术研究的发展。

致谢

本文工作部分受到比萨大学 PRA 2016 64 “Through the fog” 项目以及博岑-博尔扎诺自由大学 “Edge Cloud Orchestration” 项目的支持。

参考文献

REFERENCES

- Brogi, A., Carrasco, J., Cubo, J., D’Andria, F., Di Nitto, E., Guerriero, M., Pérez, D., Pimentel, E., and Soldani, J. (2016). SeaClouds: An Open Reference Architecture for Multi-cloud Governance, volume 9839 of LNCS, pages 334–338.
- Dupont, C., Sheikhalishahi, M., Facca, F. M., and Cretti, S. (2016). Energy efficient data centres within smart cities: IaaS and PaaS optimizations. In Smart City 360, pages 408–415.
- Felter, W., Ferreira, A., Rajamony, R., and Rubio, J. (2015). An updated performance comparison

- of virtual machines and Linux containers. In IEEE Intl Symposium On Performance Analysis of Systems and Software (ISPASS), pages 171–172.
- Jamshidi, P., Ahmad, A., and Pahl, C. (2013a). Cloud migration research: a systematic review. IEEE Transactions on Cloud Computing, 1(2):142–157.
- Jamshidi, P., Ghafari, M., Ahmad, A., and Pahl, C. (2013b). A framework for classifying and comparing architecture-centric software evolution research. In 7th European Conference on Software Maintenance and Reengineering, pages 305–314.
- Kecskemeti, G., Marosi, A. C., and Kertesz, A. (2016). The ENTICE approach to decompose monolithic services into microservices. In Intl Conf on High Performance Computing Simulation (HPCS), pages 591–596.
- Kitchenham, B., Brereton, O. P., Budgen, D., Turner, M., Bailey, J., and Linkman, S. (2009). Systematic literature reviews in software engineering – a systematic literature review. Information and Software Technology, 51(1):7–15.
- Kratzke, N. (2015). About microservices, containers and their underestimated impact on network performance. Conf on Cloud Computing, 2015.
- Lewis, J. and Fowler, M. (2014). Microservices. <http://martinfowler.com/articles/microservices.html>.
- Liu, C., Loo, B. T., and Mao, Y. (2011). Declarative automated cloud resource orchestration. In Proceedings of the 2nd ACM Symposium on Cloud Computing, page 26. ACM.
- Manu, A., Patel, J., Akhtar, S., Agrawal, V., and Murthy, K. (2016). Docker container security via heuristics-based multilateral security-conceptual and pragmatic study. In Intl Conf on Circuit, Power and Computing Techn, pages 1–14.
- Mell, P. and Grance, T. (2011). The NIST definition of cloud computing.
- Pahl, C. (2015). Containerization and the PaaS Cloud. IEEE Cloud Computing, 2(3):24–31.
- Pahl, C., Helmer, S., Miori, L., Sanin, J., and Lee, B. (2016). A container-based edge cloud PaaS architecture based on Raspberry Pi clusters. In 4th IEEE Intl Conf on Future Internet of Things and Cloud Workshops.
- Petersen, K., Feldt, R., Mujtaba, S., and Mattsson, M. (2008). Systematic mapping studies in software engineering. In Intl Conf on Evaluation and Assessment in Software Engineering.

指导教师（签字）： _____

年 月 日